

MODUL

PENGENALAN IOT :

Pembuatan Sistem Stasiun Cuaca Mini dengan Antarmuka LVGL dan Aplikasi Mobile



BY TEROBOS 2025

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya, tim penyusun dapat menyelesaikan modul pembelajaran berjudul “Pengenalan IoT: Pembuatan Sistem Stasiun Cuaca Mini dengan Antarmuka LVGL dan Aplikasi Mobile”. Modul ini disusun sebagai salah satu luaran kegiatan Program Hibah Pembelajaran Berdampak 2025, yang bertujuan untuk memperkuat kemitraan antara perguruan tinggi dan sekolah mitra dalam pengembangan pembelajaran berbasis teknologi terkini.

Modul ini dirancang sebagai bahan ajar bagi siswa Sekolah Menengah Kejuruan (SMK) dalam mengenal dan mempraktikkan konsep dasar Internet of Things (IoT). Melalui kegiatan pembelajaran berbasis proyek, peserta didik akan mempelajari bagaimana data sensor dari lingkungan dapat diolah dan ditampilkan secara real-time menggunakan mikrokontroler ESP32, antarmuka grafis LVGL, dan aplikasi mobile sederhana. Diharapkan, pengalaman belajar ini dapat menumbuhkan kemampuan teknis, logika pemrograman, serta kreativitas dalam merancang solusi digital yang bermanfaat.

Penyusunan modul ini melibatkan kolaborasi antara dosen, mahasiswa, dan guru mitra SMK sebagai bentuk nyata sinergi dunia pendidikan tinggi dan pendidikan vokasi. Modul ini terdiri atas dua bab utama, yang masing-masing membahas pembuatan antarmuka LVGL serta perakitan dan pemrograman sistem stasiun cuaca mini. Seluruh materi disusun secara sistematis dan aplikatif agar mudah diikuti oleh peserta didik tingkat menengah kejuruan.

Tim penyusun menyadari bahwa modul ini masih memiliki kekurangan, baik dari sisi isi maupun penyajian. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan modul ini di masa mendatang.

Akhir kata, semoga modul ini dapat memberikan manfaat yang luas, mendukung peningkatan kompetensi siswa SMK di bidang Internet of Things, serta menjadi salah satu kontribusi nyata dalam keberlanjutan Program Hibah Pembelajaran Berdampak 2025.

Surakarta, 11 November 2025

DAFTAR ISI

Kata Pengantar	
Daftar Isi	
Daftar Gambar	
Pengenalan Dan Dasar IoT	
1.1 Pendahuluan	
1.2 Pengenalan Perangkat Lunak Dan Framework Yang Digunakan	
1.3 Persiapan Perangkat Dan Lingkungan Pengembangan	
1.4 Pembuatan Tampilan Dan Elemen Interaktif Di Eez Studio	
1.5 Implementasi Code Di Arduino Ide	
1.6 Pengujian Dan Debug Tampilan	
1.7 Kesimpulan	
Membangun Stasiun Cuaca Mini Dan Mengirim Data Ke LVGL	
2.1 Pendahuluan	
2.2 Konsep Dan Cara Kerja Stasiun Cuaca Mini	
2.3 Perakitan Perangkat Dan Integrasi Sensor	
2.4 Pemrograman Pembacaan Data Sensor	
2.5 Pengiriman Dan Penampilan Data Pada Lvgl	
2.6 Penyegaran Dan Validasi Data Real-Time	
2.7 Kesimpulan	
Daftar Pustaka	

DAFTAR GAMBAR

Gambar 1.1. Tampilan Workspace EEZ Studio	
Gambar 1.2. Tampilan Workspace Arduino IDE	
Gambar 1.3. LVGL	
Gambar 1.4. Kabel Type C	
Gambar 1.5. Tampilan Design yang sudah dibuat	
Gambar 2.1. Data Flow Sistem	
Gambar 2.2. Sensor LDR	
Gambar 2.3. Sensor DHT11	
Gambar 2.4. Sensor Raindrop	
Gambar 2.5. Kabel Jumper	
Gambar 2.6. Rangkaian Perangkat Stasiun Cuaca Mini	

TEROBOS

CHAPTER 1

Pengenalan dan Dasar IoT

1.1 Pendahuluan

1.1.1 Latar Belakang

Seiring dengan perkembangan teknologi Internet of Things (IoT) dan sistem embedded, kebutuhan akan antarmuka pengguna (User Interface) yang informatif dan mudah dipahami menjadi semakin penting. Pada masa digitalisasi saat ini, masyarakat sangat membutuhkan sebuah perangkat yang praktis dan mudah digunakan untuk mendapatkan informasi tentang cuaca yang akurat (Sani & Firdaus, 2021). Dalam aplikasi monitoring lingkungan seperti stasiun cuaca, visualisasi data yang baik tidak hanya memudahkan pengguna dalam memahami informasi, tetapi juga meningkatkan pengalaman pengguna secara keseluruhan.

LVGL (Light and Versatile Graphics Library) hadir sebagai solusi untuk mengembangkan antarmuka grafis yang menarik dan responsif pada sistem embedded dengan resource yang terbatas. Library ini telah banyak digunakan dalam berbagai aplikasi IoT, perangkat wearable, hingga sistem kontrol industri karena kemampuannya menghasilkan tampilan yang modern dengan konsumsi memori yang efisien.

ESP32, sebagai mikrokontroler populer dengan kemampuan WiFi dan Bluetooth terintegrasi, menjadi platform yang ideal untuk proyek stasiun cuaca mini. Kombinasi ESP32 dengan display TFT dan LVGL memungkinkan kita untuk membuat sistem monitoring yang tidak hanya fungsional, tetapi juga memiliki tampilan yang profesional dan user-friendly.

Dalam modul ini, kita akan mengintegrasikan sensor suhu, cahaya, dan hujan dengan antarmuka grafis berbasis LVGL. Proses desain akan dipermudah dengan menggunakan EEZ Studio, sebuah tool visual yang memungkinkan kita merancang UI secara drag-and-drop, kemudian mengeksport hasilnya ke kode yang siap digunakan di Arduino IDE. Pendekatan ini tidak hanya mempercepat proses pengembangan, tetapi juga memungkinkan iterasi desain yang lebih cepat tanpa harus menulis kode UI secara manual.

Proyek stasiun cuaca mini ini akan menjadi contoh praktis bagaimana teknologi embedded modern dapat digunakan untuk membuat sistem monitoring yang lengkap, dari sensor hardware hingga visualisasi data yang intuitif.

1.1.2 Tujuan Pembelajaran

Setelah menyelesaikan modul ini, peserta diharapkan mampu:

- Memahami arsitektur LVGL dan cara kerjanya pada ESP32
- Menggunakan EEZ Studio untuk merancang antarmuka grafis secara visual
- Mengintegrasikan sensor suhu, cahaya, dan hujan dengan ESP32
- Mengimplementasikan tampilan data real-time pada display TFT
- Mengelola event handler dan interaksi pengguna pada LVGL

- Melakukan testing, debugging, dan optimasi aplikasi embedded dengan GUI
- Mengembangkan sistem stasiun cuaca mini yang fungsional dengan tampilan profesional

1.2 Pengenalan Perangkat Lunak dan Framework yang Digunakan

1.2.1 Pengenalan LVGL dan Arsitekturnya di ESP32

LVGL (Light and Versatile Graphics Library) adalah library grafis open-source yang dirancang khusus untuk sistem embedded dengan resource terbatas. Library ini memungkinkan pengembang untuk membuat antarmuka pengguna (User Interface) yang modern, responsif, dan menarik secara visual pada perangkat mikrokontroler seperti ESP32, STM32, Arduino, dan platform embedded lainnya. Berbeda dengan library GUI konvensional yang dirancang untuk sistem komputer dengan resource melimpah, LVGL dioptimalkan secara khusus untuk bekerja efisien pada perangkat dengan RAM terbatas, processing power rendah, dan display yang bervariasi.

Dikembangkan pertama kali oleh Gabor Kiss-Vamosi pada tahun 2016, LVGL telah berkembang menjadi salah satu library GUI paling populer untuk sistem embedded. Library ini ditulis dalam bahasa C untuk memastikan portabilitas maksimal dan dapat berjalan pada berbagai platform dengan minimal modifikasi. Hingga saat ini, LVGL telah digunakan dalam ribuan produk komersial dan proyek open-source di seluruh dunia, mulai dari perangkat wearable, sistem kontrol industri, hingga aplikasi IoT. Kepopuleran LVGL tidak lepas dari filosofi desainnya yang mengutamakan keseimbangan antara performa, fitur, dan efisiensi resource.

LVGL bersifat platform-independent, artinya kode yang ditulis untuk satu platform dapat dengan mudah dipindahkan ke platform lain tanpa perubahan signifikan. Hal ini dimungkinkan melalui Hardware Abstraction Layer (HAL) yang memisahkan logic aplikasi dari detail implementasi hardware. Library ini juga bersifat modular, dimana developer dapat memilih fitur-fitur yang dibutuhkan dan mengexclude fitur yang tidak diperlukan untuk menghemat memori. Fleksibilitas ini sangat penting dalam dunia embedded dimana setiap kilobyte memori berharga.

Salah satu aspek yang membuat LVGL menonjol adalah fokusnya pada developer experience. Library ini menyediakan API yang konsisten dan mudah dipahami, dokumentasi yang komprehensif dengan banyak contoh kode, serta tool-tool pendukung seperti simulator dan visual designer. LVGL juga mendukung berbagai gaya pemrograman, mulai dari procedural programming hingga event-driven architecture, memberikan kebebasan kepada developer untuk memilih pendekatan yang paling sesuai dengan kebutuhan proyek mereka. Ekosistem yang lengkap ini menjadikan LVGL bukan hanya sebagai library GUI, tetapi sebagai framework lengkap untuk pengembangan aplikasi embedded dengan interface grafis.

LVGL memiliki tiga karakteristik utama yang menjadikannya pilihan ideal untuk pengembangan GUI pada sistem embedded.

1. Lightweight (ringan), dimana LVGL hanya membutuhkan RAM minimal 64 KB untuk operasi dasar dan flash memory minimum 64 KB untuk library dan font. Optimasi memori yang sangat efisien memungkinkan LVGL berjalan pada

mikrokontroler dengan resource terbatas tanpa mengorbankan kualitas tampilan.

2. Versatile (serbaguna), yang tercermin dari dukungan LVGL terhadap berbagai jenis display seperti TFT LCD, OLED, dan E-ink, serta kompatibilitas dengan berbagai resolusi layar dari 320x240 hingga Full HD. LVGL dapat bekerja dengan atau tanpa sistem operasi (RTOS) dan mendukung berbagai input device seperti touchscreen, keyboard, mouse, dan rotary encoder. Fleksibilitas ini memungkinkan LVGL diimplementasikan pada hampir semua jenis hardware embedded.
3. Feature-rich (kaya fitur), dimana LVGL menyediakan lebih dari 40 widget siap pakai seperti button, label, slider, chart, dan gauge. Library ini dilengkapi dengan sistem styling yang fleksibel mirip CSS, animasi dan transisi yang smooth, support multi-bahasa dan font custom, serta anti-aliasing untuk menghasilkan tampilan yang lebih halus dan profesional. Kombinasi fitur-fitur ini memungkinkan pembuatan interface yang kompleks dan interaktif dengan effort yang minimal.

1.2.1.1 Arsitektur LVGL

LVGL memiliki arsitektur modular yang terdiri dari tiga layer utama yang bekerja secara hierarkis. Layer paling bawah adalah Hardware Abstraction Layer (HAL), yang berfungsi sebagai jembatan antara LVGL dan hardware spesifik. HAL mengatur display driver untuk pengiriman data ke layar, mengelola input device driver seperti touchscreen dan button, serta memungkinkan LVGL berjalan pada berbagai platform tanpa perlu modifikasi pada core library. Abstraksi ini sangat penting untuk portabilitas LVGL ke berbagai jenis hardware.

Di atas HAL terdapat Core Layer yang merupakan inti dari LVGL. Layer ini bertanggung jawab mengelola object (widget) dan hierarchy-nya, mengatur rendering dan refresh display, menangani event dan input processing, serta melakukan memory management. Core Layer memastikan bahwa semua operasi GUI berjalan secara efisien dan responsif, bahkan pada hardware dengan keterbatasan processing power.

Layer teratas adalah Widget Layer yang menyediakan berbagai komponen UI siap pakai. Setiap widget dapat dikustomisasi sepenuhnya sesuai kebutuhan desain, dan LVGL mendukung nested widget dimana widget dapat ditempatkan di dalam widget lainnya untuk membuat struktur UI yang kompleks. Widget Layer memanfaatkan fungsi-fungsi dari Core Layer dan HAL untuk merender tampilan yang interaktif dan responsif.

1.2.1.2 Keunggulan LVGL untuk Embedded Systems

LVGL menawarkan beberapa keunggulan signifikan yang menjadikannya pilihan unggul untuk pengembangan aplikasi embedded. Dari segi efisiensi resource, LVGL dirancang untuk bekerja optimal pada hardware dengan keterbatasan melalui teknik partial rendering dan buffering yang cerdas. Teknik ini memungkinkan LVGL

menghasilkan animasi yang smooth bahkan pada MCU dengan clock speed rendah, tanpa membebani CPU atau memori secara berlebihan.

Kemudahan penggunaan menjadi keunggulan berikutnya, dimana API LVGL yang intuitif dan dokumentasi yang lengkap membuat library ini mudah dipelajari, bahkan untuk pemula sekalipun. Ketersediaan tool visual seperti SquareLine Studio dan EEZ Studio semakin mempermudah proses desain UI dengan pendekatan drag-and-drop, mengurangi kebutuhan untuk menulis kode secara manual.

LVGL juga mampu menghasilkan tampilan modern yang profesional. Dengan dukungan anti-aliasing, gradasi warna, shadow, dan animasi, LVGL dapat menciptakan interface yang setara dengan aplikasi smartphone modern. Kualitas visual ini sangat penting untuk meningkatkan user experience, terutama pada produk komersial yang membutuhkan tampilan menarik.

Keunggulan terakhir adalah komunitas yang aktif dan terus berkembang. LVGL memiliki komunitas developer yang besar di berbagai platform seperti GitHub, Forum resmi, dan Discord. Komunitas ini menyediakan tutorial, contoh proyek, dan solusi untuk berbagai masalah yang mungkin dihadapi developer. Support dari komunitas membuat proses learning dan troubleshooting menjadi lebih mudah dan cepat.

1.2.1.3 Aplikasi LVGL dalam Proyek Stasiun Cuaca

Dalam proyek stasiun cuaca mini ini, LVGL akan digunakan untuk menciptakan dashboard monitoring yang informatif dan interaktif. Data suhu akan ditampilkan dalam bentuk angka dengan font yang besar dan jelas, serta divisualisasikan dalam bentuk grafik trend untuk melihat perubahan suhu dari waktu ke waktu. Intensitas cahaya akan direpresentasikan menggunakan gauge atau bar indicator yang memberikan gambaran visual intuitif tentang kondisi pencahayaan lingkungan.

Status hujan akan ditampilkan melalui icon yang menarik dengan animasi untuk memberikan feedback visual yang lebih engaging. Dashboard akan dirancang dengan prinsip information hierarchy yang baik, memastikan informasi penting mudah dibaca dan dipahami dalam sekejap. Kontrol interaktif seperti button untuk mengubah mode tampilan atau reset nilai juga akan diimplementasikan, memanfaatkan event handling system LVGL yang powerful.

Dengan memanfaatkan LVGL, stasiun cuaca mini ini tidak hanya berfungsi sebagai alat monitoring data mentah, tetapi juga sebagai sistem informasi visual yang menyajikan data dalam format yang mudah dipahami, menarik untuk dilihat, dan nyaman untuk digunakan. Pendekatan visual yang baik akan meningkatkan user experience secara signifikan, menjadikan stasiun cuaca ini tidak hanya fungsional tetapi juga menyenangkan untuk digunakan sehari-hari.

1.2.2 Pengenalan EEZ Studio

EEZ Studio adalah Integrated Development Environment (IDE) open-source yang dirancang khusus untuk membantu pengembang membuat antarmuka pengguna (User Interface) pada sistem embedded secara visual. Studio ini

dikembangkan oleh Envoy d.o.o. dan menyediakan pendekatan WYSIWYG (What You See Is What You Get) dalam mendesain GUI, sehingga pengembang dapat melihat hasil desain secara real-time tanpa harus menulis kode secara manual. EEZ Studio mendukung berbagai library GUI termasuk LVGL, yang menjadikannya tool yang ideal untuk proyek stasiun cuaca mini kita.

Berbeda dengan pendekatan tradisional dimana developer harus menulis ratusan baris kode untuk membuat interface sederhana, EEZ Studio mengubah paradigma pengembangan GUI embedded dengan menyediakan visual editor yang intuitif. Tool ini memungkinkan desainer dan programmer bekerja secara kolaboratif, dimana desainer dapat fokus pada aspek visual dan user experience, sementara programmer dapat fokus pada logic aplikasi dan integrasi sensor. Pemisahan concern ini tidak hanya mempercepat development cycle, tetapi juga meningkatkan kualitas hasil akhir karena setiap aspek ditangani oleh expertise yang tepat.

EEZ Studio juga mendukung workflow iteratif yang sangat penting dalam pengembangan produk. Perubahan desain dapat dilakukan dengan cepat melalui visual editor, kemudian langsung di-test melalui simulator built-in sebelum di-deploy ke hardware. Proses iterasi yang cepat ini memungkinkan eksperimen dengan berbagai desain alternatif tanpa membuang waktu untuk recompile dan upload ke device. Ketika desain sudah final, EEZ Studio dapat menggenerate kode yang clean, well-structured, dan siap untuk diintegrasikan dengan sistem yang lebih besar.

1.2.2.1 Fitur Utama EEZ Studio

1. Visual Editor

EEZ Studio dilengkapi dengan visual editor yang menyediakan interface drag-and-drop intuitif, memungkinkan penambahan dan pengaturan widget dengan mudah. Preview real-time memberikan feedback instan terhadap setiap perubahan yang dibuat, sementara property editor yang komprehensif memungkinkan konfigurasi detail untuk setiap widget mulai dari ukuran, posisi, warna, hingga behavior. Tool ini juga mendukung multiple screens dan navigation system, memungkinkan pembuatan aplikasi multi-halaman yang kompleks dengan transisi antar screen yang smooth.

2. LVGL Integration

Integrasi LVGL yang dimiliki EEZ Studio sangat mendalam dan native. Studio ini menyediakan support penuh untuk semua widget LVGL, termasuk widget-widget advanced seperti chart, meter, dan animview. Automatic code generation menghasilkan kode C/C++ yang optimized dan readable, kompatibel dengan LVGL versi 8 dan 9. Preview yang ditampilkan di EEZ Studio menggunakan rendering engine LVGL yang sama dengan target hardware, memastikan "what you see is what you get" secara akurat tanpa surprises saat deployment.

3. Asset Management

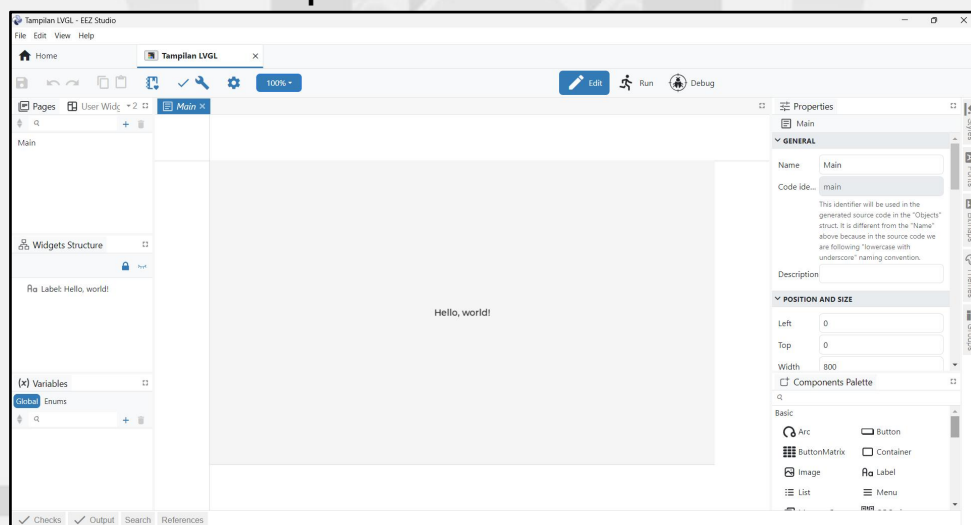
Asset management di EEZ Studio dirancang untuk mempermudah pengelolaan resource grafis. Image converter built-in mendukung berbagai format (PNG, JPG, BMP) dan secara otomatis mengoptimalkan untuk

embedded systems. Font converter memungkinkan penggunaan custom font dengan subset character yang dapat dikonfigurasi untuk menghemat memori. Color format optimization memastikan image menggunakan format yang paling efisien sesuai dengan capability display target. Semua asset terorganisir dalam struktur hirarki yang jelas, memudahkan management pada project besar dengan ratusan resource.

4. Simulator

Simulator yang terintegrasi dalam EEZ Studio memungkinkan testing UI langsung tanpa memerlukan hardware fisik. Developer dapat mensimulasikan touchscreen interaction, button press, dan berbagai input lainnya untuk memvalidasi behavior aplikasi. Debug mode menyediakan inspeksi real-time terhadap widget properties dan state, membantu troubleshooting issue sebelum deployment. Performance monitoring menampilkan metrics seperti frame rate dan memory usage, membantu identifikasi bottleneck sejak awal development cycle.

1.2.2.2 Interface dan Workspace EEZ Studio



Gambar 1.1. Tampilan Workspace EEZ Studio

Workspace EEZ Studio dirancang dengan ergonomis untuk memaksimalkan produktivitas developer. Berikut adalah komponen-komponen utama dari interface EEZ Studio:

1. Project Explorer

Project Explorer di sisi kiri menampilkan struktur project secara hierarki, termasuk screens, styles, fonts, images, dan assets lainnya. Navigasi antar komponen dapat dilakukan dengan cepat melalui tree view yang intuitif, dan drag-drop reorganization memungkinkan restructuring project dengan mudah.

2. Canvas/Editor Area

Canvas atau Editor Area menempati bagian tengah workspace sebagai area utama untuk mendesain UI. Grid system dan snapping guides membantu alignment yang presisi, sementara zoom controls memungkinkan detail work pada level pixel atau overview pada level layout. Multi-screen editing

memungkinkan bekerja pada beberapa screen secara simultan, berguna untuk menjaga konsistensi visual antar halaman.

3. Widgets Palette

Widgets Palette biasanya terletak di sisi kanan, menampilkan daftar lengkap widget LVGL yang tersedia di kategorisasi berdasarkan fungsi seperti Basic, Display, Input, Navigation, dan Visualization. Quick search memudahkan pencarian widget spesifik, dan recently used widgets dapat di-access dengan cepat untuk mempercepat workflow. Setiap widget dapat di-drag langsung ke kanvas untuk instant placement.

4. Properties Panel

Properties Panel menyediakan kontrol detail untuk widget yang sedang dipilih. Panel ini terbagi dalam beberapa section seperti Basic (size, position), Style (colors, fonts, borders), States (default, checked, disabled), dan Events. Perubahan pada properties langsung terefleksikan di kanvas, memberikan feedback visual yang instan. Advanced properties untuk widget kompleks seperti chart dapat dikonfigurasi melalui dedicated editors dengan interface yang lebih spesifik.

5. Outline/Hierarchy Panel

Outline atau Hierarchy Panel menampilkan struktur parent-child relationship antar widget dalam bentuk tree. Panel ini sangat berguna untuk navigasi pada UI yang kompleks dengan banyak nested elements, dan memungkinkan reordering widget dengan drag-drop untuk mengubah z-order atau hierarchy.

1.2.2.3 Keunggulan EEZ Studio dalam Development

1. Kecepatan Development

Visual editor mengeliminasi kebutuhan menulis boilerplate code untuk setiap widget, menghemat waktu yang dapat digunakan untuk fokus pada logic aplikasi. Iterasi desain yang biasanya memakan waktu berjam-jam dalam traditional coding dapat dilakukan dalam hitungan menit dengan visual editor. Time-to-market yang lebih cepat memberikan competitive advantage, terutama dalam industri yang fast-paced.

2. Pengurangan Error

Code generation otomatis tidak hanya mempercepat development, tetapi juga mengurangi error secara drastis. Manual coding GUI sering menghasilkan typo, syntax error, atau inconsistent code structure yang memakan waktu untuk debug. EEZ Studio menghasilkan kode yang consistent, mengikuti best practices, dan sudah teruji. Memory leaks dan resource management issues yang sering terjadi dalam manual GUI coding dapat dihindari karena generated code sudah dioptimasi.

3. Kemudahan Pembelajaran

Developer yang baru mengenal LVGL dapat langsung produktif dengan EEZ Studio tanpa harus menguasai seluruh API LVGL terlebih dahulu. Visual representation membuat konsep-konsep seperti widget hierarchy, styling, dan event handling lebih mudah dipahami dibanding membaca dokumentasi text.

Learning curve yang lebih gentle membuat onboarding team member baru menjadi lebih cepat.

4. Maintainability yang Lebih Baik

Separation of concerns yang difasilitasi EEZ Studio meningkatkan maintainability code. UI definition terpisah dari application logic, membuat perubahan pada salah satu aspek tidak mempengaruhi yang lain. Ketika designer ingin mengubah warna atau layout, programmer tidak perlu terlibat dan vice versa. Generated code juga lebih mudah di-review dan di-maintain karena strukturnya yang konsisten dan predictable.

1.2.3 Pengenalan Arduino IDE

Arduino IDE (Integrated Development Environment) adalah software open-source yang digunakan untuk menulis, mengcompile, dan mengupload program ke board mikrokontroler Arduino dan compatible boards seperti ESP32. Dikembangkan pertama kali oleh Arduino Team pada tahun 2005, IDE ini dirancang dengan filosofi "easy to use" untuk membuat pemrograman embedded systems dapat diakses oleh siapa saja, dari pemula hingga profesional developer. Arduino IDE menyediakan interface yang sederhana namun powerful, dengan fitur-fitur yang cukup untuk pengembangan aplikasi embedded yang kompleks.

Arduino IDE menggunakan bahasa pemrograman berbasis C/C++ dengan library framework yang menyederhanakan banyak operasi hardware level. Framework Arduino mengabstraksi kompleksitas register manipulation dan hardware configuration menjadi fungsi-fungsi high-level yang mudah dipahami seperti `digitalWrite()`, `analogRead()`, dan `Serial.println()`. Pendekatan ini menurunkan barrier to entry secara signifikan, memungkinkan fokus pada logic aplikasi daripada detail teknis hardware.

Meskipun awalnya dikembangkan untuk board Arduino official, IDE ini telah berkembang menjadi platform universal untuk berbagai mikrokontroler melalui sistem Board Manager. ESP32, STM32, Raspberry Pi Pico, dan puluhan platform lainnya dapat diprogram menggunakan Arduino IDE dengan menginstall board support package yang sesuai. Ekosistem yang luas ini, dikombinasikan dengan ribuan library yang tersedia melalui Library Manager, menjadikan Arduino IDE sebagai pilihan populer untuk rapid prototyping dan production development.

1.2.3.1 Fitur Utama Arduino IDE

1. Code Editor

Arduino IDE dilengkapi dengan code editor yang menyediakan syntax highlighting untuk memudahkan pembacaan kode, dengan warna berbeda untuk keywords, variables, functions, dan comments. Auto-completion membantu mempercepat penulisan kode dengan menyarankan fungsi dan variabel yang tersedia. Line numbering dan code folding memudahkan navigasi pada file yang besar, sementara find and replace function dengan support regex memungkinkan refactoring code secara efisien.

2. Compiler dan Upload System

Compiler dan upload system terintegrasi memungkinkan one-click compilation dan deployment ke hardware. Arduino IDE menggunakan avr-gcc, arm-gcc, atau xtensa-gcc tergantung target board, dan secara otomatis menangani kompleksitas toolchain configuration. Verbose output mode menyediakan informasi detail tentang proses compilation dan upload, berguna untuk troubleshooting. Over-the-air (OTA) upload support untuk ESP32 memungkinkan wireless programming tanpa koneksi fisik.

3. Serial Monitor

Serial Monitor adalah tool essential untuk debugging dan komunikasi dengan board. Monitor ini menampilkan data yang dikirim dari mikrokontroler melalui serial port, memungkinkan real-time observation terhadap program behavior. Baud rate dapat dikonfigurasi sesuai kebutuhan, dan fitur autoscroll memastikan data terbaru selalu visible. Serial Plotter menyediakan visualisasi grafis untuk data numerik, sangat berguna untuk monitoring sensor readings atau signal processing.

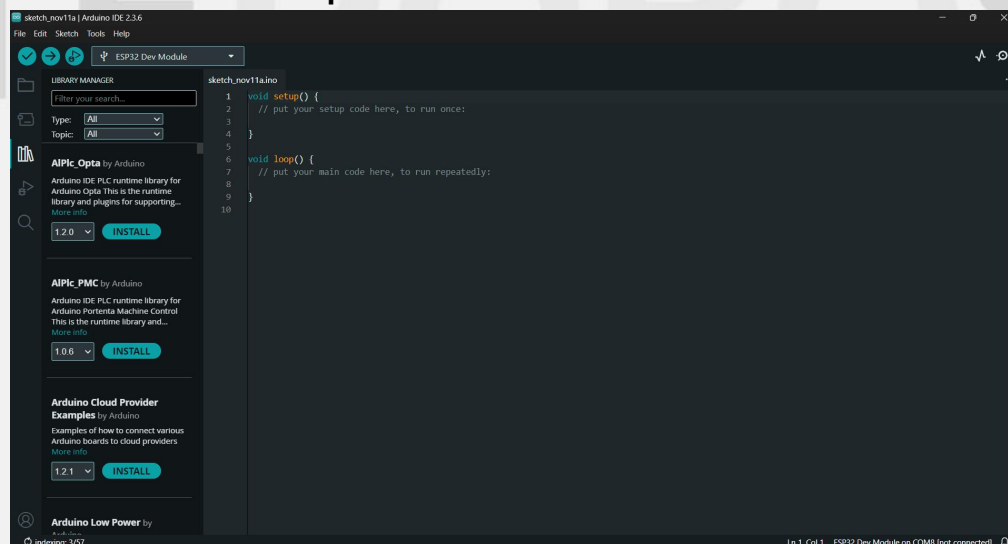
4. Library Manager

Library Manager menyederhanakan dependency management dengan menyediakan akses ke ribuan library community-contributed. Library dapat di-search, di-install, dan di-update langsung dari IDE tanpa manual download. Dependency resolution otomatis memastikan semua library yang dibutuhkan terinstall dengan versi yang compatible. Custom library juga dapat ditambahkan dengan menempatkannya di folder libraries.

5. Board Manager

Board Manager memungkinkan instalasi support untuk berbagai platform mikrokontroler. ESP32 support, misalnya, dapat diinstall dengan beberapa klik, memberikan akses ke semua pin definitions, libraries, dan examples yang spesifik untuk ESP32. Multiple board definitions dapat diinstall secara simultan, memudahkan development untuk multi-platform projects.

1.2.3.2 Interface dan Workspace Arduino IDE



Gambar 1.2. Tampilan Workspace Arduino IDE

Arduino IDE memiliki interface yang minimalis namun fungsional, dirancang untuk mengurangi distraction dan memaksimalkan screen space untuk coding. Berikut adalah komponen utama dari interface Arduino IDE:

1. Menu Bar

Menu Bar di bagian atas menyediakan akses ke semua fungsi IDE melalui menu File, Edit, Sketch, Tools, dan Help. Menu File menangani operasi file seperti New, Open, Save, dan Examples. Menu Sketch berisi fungsi untuk Verify/Compile, Upload, dan Export Compiled Binary. Menu Tools menyediakan akses ke Board Manager, Library Manager, Serial Monitor, dan Serial Plotter.

2. Toolbar

Toolbar berisi shortcut button untuk fungsi yang sering digunakan. Tombol Verify (checkmark icon) meng-compile sketch tanpa upload untuk quick syntax checking. Tombol Upload (arrow icon) meng-compile dan mengupload ke board. Tombol New, Open, dan Save menyediakan quick access untuk file operations. Tombol Serial Monitor membuka jendela komunikasi dengan board.

3. Editor Area

Editor Area menempati mayoritas screen space, menyediakan environment untuk menulis code. Syntax highlighting dengan color coding memudahkan identifikasi berbagai elemen code. Tab system memungkinkan multiple files terbuka secara simultan, berguna untuk projects dengan multiple source files. Gutter di sisi kiri menampilkan line numbers untuk easy reference.

4. Console/Output Area

Console Area di bagian bawah menampilkan output dari compilation dan upload process. Pesan error ditampilkan dengan detail termasuk line number dan deskripsi masalah, memudahkan debugging. Warning messages memberikan informasi tentang potential issues yang tidak menghalangi compilation. Verbose mode dapat diaktifkan untuk melihat detail lengkap proses compilation.

5. Status Bar

Status Bar di bagian paling bawah menampilkan informasi tentang board yang sedang dipilih dan port yang digunakan. Memory usage ditampilkan setelah compilation, menunjukkan berapa persen flash dan RAM yang terpakai. Status ini penting untuk memastikan program fit dalam available memory.

1.2.3.3 Keunggulan Arduino IDE untuk Pengembangan ESP32

1. Simplicity dan Ease of Use

Arduino IDE dirancang dengan prinsip simplicity, menghilangkan kompleksitas yang tidak perlu dari embedded programming. Setup dan configuration minimal diperlukan untuk memulai development, developer dapat langsung fokus pada writing code. Interface yang clean dan

straightforward mengurangi cognitive load, terutama beneficial untuk beginners atau untuk rapid prototyping sessions.

2. Extensive Library Ecosystem

Library ecosystem Arduino yang massive menyediakan solusi ready-made untuk hampir semua kebutuhan. Untuk proyek stasiun cuaca, library untuk sensor DHT, BH1750, dan rain sensor sudah tersedia dan well-documented. LVGL library dengan ESP32 support memungkinkan seamless integration dengan GUI yang dibuat di EEZ Studio. Networking libraries untuk WiFi, HTTP, dan MQTT memudahkan implementasi IoT features.

3. Active Community Support

Arduino memiliki salah satu komunitas embedded systems terbesar di dunia. Forum Arduino, Reddit communities, dan countless blogs menyediakan wealth of knowledge dan troubleshooting resources. Untuk hampir setiap error atau challenge, kemungkinan besar seseorang sudah mengalami dan membagikan solusinya. Community-contributed examples dan projects memberikan starting point dan inspiration.

4. Cross-Platform Compatibility

Arduino IDE berjalan di Windows, macOS, dan Linux tanpa perbedaan signifikan dalam functionality. Ini memungkinkan team collaboration dimana members menggunakan operating system berbeda. Sketch files dan libraries portable antar platform, ensuring consistency dalam development environment.

5. Fast Iteration Cycle

Compile dan upload cycle di Arduino IDE sangat fast, biasanya hanya beberapa detik untuk project berukuran medium. Fast iteration memungkinkan rapid testing dan debugging, accelerating development process. Serial Monitor yang instant-access memberikan immediate feedback, memudahkan verification bahwa code changes berfungsi seperti intended.

1.3 Persiapan Perangkat dan Lingkungan Pengembangan

Sebelum memulai pengembangan stasiun cuaca mini, persiapan yang matang terhadap perangkat keras (hardware) dan perangkat lunak (software) sangat penting untuk memastikan proses development berjalan lancar. Bagian ini akan menjelaskan secara detail semua komponen hardware yang dibutuhkan beserta fungsinya, serta software yang harus diinstall untuk mendukung pengembangan proyek. Dengan persiapan yang tepat, waktu troubleshooting dapat diminimalkan dan fokus dapat diarahkan pada implementasi fitur dan optimasi sistem.

1.3.1 Hardware yang dibutuhkan

Komponen hardware untuk proyek stasiun cuaca mini ini dipilih berdasarkan ketersediaan, kemudahan penggunaan, dan kompatibilitas dengan ESP32 dan LVGL. Setiap komponen memiliki peran spesifik dalam sistem, dan pemahaman yang baik terhadap fungsi masing-masing komponen akan memudahkan proses wiring dan programming.

1. TFT LCD Display dengan Touchscreen



Gambar 1.3. LVGL

Sumber: [TFT LCD Display dengan Touch Pen](#)

Display yang digunakan adalah TFT LCD dengan driver ST7789 berukuran 3.2 inch dengan resolusi 320x240 pixels. Display ini dilengkapi dengan touchscreen resistive yang memungkinkan interaksi pengguna dengan antarmuka. TFT LCD ini menggunakan interface SPI (Serial Peripheral Interface) untuk komunikasi dengan ESP32, yang menawarkan kecepatan transfer data yang cukup untuk rendering grafis LVGL dengan smooth.

Driver ST7789 telah widely supported oleh berbagai library Arduino, termasuk TFT_eSPI yang akan kita gunakan. Resolusi 320x240 pixels memberikan area display yang cukup untuk menampilkan multiple widgets seperti gauge, label, chart, dan button secara bersamaan tanpa terlihat cramped. Touchscreen resistive memungkinkan input dari stylus atau jari, memberikan fleksibilitas dalam user interaction.

Display ini biasanya sudah terintegrasi dengan ESP32 dalam satu board development, sehingga koneksi hardware sudah pre-wired dan tinggal melakukan konfigurasi software. Jika menggunakan display terpisah, perlu memperhatikan pin connections untuk SPI (MOSI, MISO, SCK, CS, DC, RST) dan touchscreen controller (T_CLK, T_CS, T_DIN, T_DO, T_IRQ).

2. Kabel USB Type-C



Gambar 1.4. Kabel Type C

Sumber: [Kabel Type-C](#)

Kabel USB Type-C digunakan untuk menghubungkan TFT LCD display (yang sudah terintegrasi dengan ESP32) ke laptop atau komputer. Kabel ini memiliki dual function: sebagai power supply untuk board dan sebagai komunikasi interface untuk programming dan serial debugging. Koneksi USB-to-UART bridge chip pada board (biasanya CP2102 atau CH340) mengkonversi USB signal menjadi serial UART yang dipahami oleh ESP32.

Kualitas kabel USB sangat berpengaruh pada stability programming dan power delivery. Kabel USB yang murahan sering hanya memiliki power wires tanpa data wires, atau memiliki resistansi tinggi yang menyebabkan voltage drop. Disarankan menggunakan kabel USB Type-C yang berkualitas dengan data wires yang proper dan panjang tidak lebih dari 1.5 meter untuk meminimalkan signal degradation.

1.3.2 Software yang dibutuhkan

Software development environment untuk proyek stasiun cuaca mini terdiri dari dua komponen utama: Arduino IDE untuk programming dan compilation, serta EEZ Studio untuk visual GUI design. Kedua software ini akan bekerja secara complementary, dimana EEZ Studio menggenerate UI code yang kemudian diintegrasikan dan di-compile di Arduino IDE bersama dengan sensor logic dan system code.

1. Instalasi dan Konfigurasi Arduino IDE

a. Download dan Instalasi Arduino IDE

- 1) Buka browser dan akses website resmi Arduino di <https://www.arduino.cc/en/software>
- 2) Pilih versi Arduino IDE sesuai sistem operasi yang digunakan (Windows, macOS, atau Linux)
- 3) Download installer:
 - Untuk Windows: pilih "Windows Win 10 and newer" untuk installer atau "Windows ZIP file" untuk portable version

- Untuk macOS: download file .dmg
 - Untuk Linux: download ApplImage atau gunakan package manager
- 4) Jalankan installer dan ikuti instruksi instalasi:
 - Windows: double-click installer dan ikuti wizard, centang opsi "Install USB driver" jika diminta
 - macOS: drag aplikasi Arduino IDE ke folder Applications
 - Linux: berikan permission executable dengan `chmod +x` kemudian jalankan
 - 5) Setelah instalasi selesai, buka Arduino IDE untuk memverifikasi instalasi berhasil
- b. Instalasi ESP32 Board Support Package
- ESP32 board support package diperlukan agar Arduino IDE dapat recognize dan compile code untuk ESP32. Berikut langkah instalasinya:
- 1) Buka Arduino IDE
 - 2) Masuk ke menu File > Preferences (atau Arduino IDE > Settings di macOS)
 - 3) Pada bagian "Additional Boards Manager URLs", masukkan URL berikut:

https://dl.espressif.com/dl/package_esp32_index.json

(Jika sudah ada URL lain, pisahkan dengan koma atau klik icon di samping untuk membuka dialog dan tambahkan di baris baru)
 - 4) Klik OK untuk menyimpan preferences
 - 5) Buka Tools > Board > Boards Manager
 - 6) Pada search box, ketik "esp32"
 - 7) Cari package "Arduino ESP32 Boards by Arduino" dan "esp32 by Espressif Systems"
 - 8) Klik tombol Install untuk kedua package tersebut (pilih versi terbaru atau versi stable yang direkomendasikan)
 - 9) Tunggu hingga proses download dan instalasi selesai (ukuran package cukup besar, sekitar 250-300 MB)
 - 10) Setelah instalasi selesai, close Boards Manager
 - 11) Verifikasi instalasi dengan masuk ke Tools > Board dan pastikan berbagai varian ESP32 muncul dalam list (ESP32 Dev Module, ESP32-WROOM-DA Module, dll)
- c. Konfigurasi Board ESP32
- Setelah board support package terinstall, lakukan konfigurasi board settings:
- 1) Hubungkan ESP32 (TFT Display yang terintegrasi dengan ESP32) ke laptop menggunakan kabel USB Type-C
 - 2) Pada Arduino IDE, masuk ke Tools > Board dan pilih board ESP32 yang sesuai:

- Jika menggunakan generic ESP32 development board: pilih ESP32 Dev Module
 - Jika menggunakan board khusus (seperti ESP32-WROOM-32): pilih board type yang sesuai
- 3) Pilih COM Port di Tools > Port:
- Windows: akan muncul sebagai "COM3", "COM4", dst
 - macOS: akan muncul sebagai "/dev/cu.usbserial-xxxxx"
 - Linux: akan muncul sebagai "/dev/ttyUSB0" atau "/dev/ttyACM0"
 - Jika port tidak muncul, install driver USB (CP210x atau CH340) sesuai chip yang digunakan board
- 4) Konfigurasi settings tambahan di menu Tools:
- Upload Speed: 115200 (atau 921600 untuk upload lebih cepat, tapi bisa kurang stable)
 - CPU Frequency: 240MHz (untuk performa maksimal)
 - Flash Frequency: 80MHz
 - Flash Mode: QIO
 - Flash Size: sesuai ESP32 yang digunakan (biasanya 4MB)
 - Partition Scheme: "Default 4MB with spiiffs" atau "Minimal SPIFFS"
 - Core Debug Level: "None" (ubah ke "Verbose" jika perlu debugging detail)
- 5) Test koneksi dengan mengupload sketch sederhana:
- Buka File > Examples > 01.Basics > Blink
 - Klik tombol Upload (ikon panah ke kanan)
 - Tunggu hingga proses compile dan upload selesai
 - Jika berhasil, LED built-in pada ESP32 akan berkedip
- d. Instalasi Library yang Dibutuhkan
- Library-library berikut harus diinstall untuk mendukung pengembangan stasiun cuaca:
- 1) LVGL (Light and Versatile Graphics Library)
- Buka Tools > Manage Libraries atau Sketch > Include Library > Manage Libraries
 - Pada search box, ketik "lvgl"
 - Cari library "lvgl by kishogin"
 - Pilih versi yang compatible dengan EEZ Studio (biasanya versi 8.x atau 9.x). Tapi untuk proyek ini lebih baik menggunakan versi 8.x
 - Klik Install
 - Jika diminta install dependencies, klik Install All
 - Setelah instalasi selesai, lakukan konfigurasi LVGL:
 - Buka folder library LVGL (biasanya di Documents/Arduino/libraries/lvgl)

- Copy file lv_conf_template.h dan rename menjadi lv_conf.h
- Pindahkan lv_conf.h ke folder Arduino libraries (satu level di atas folder lvgl)
- Edit lv_conf.h:
 - Ubah #if 0 menjadi #if 1 di baris pertama
 - Sesuaikan konfigurasi memory dan display sesuai kebutuhan (akan dijelaskan detail di bab implementasi)

2) TFT_eSPI (Display Driver Library)

- Di Library Manager, search "TFT_eSPI"
- Cari library "TFT_eSPI by Bodmer"
- Klik Install

3) DHT Sensor Library

- Di Library Manager, search "DHT sensor library"
- Cari library "DHT sensor library by Adafruit"
- Klik Install
- Jika diminta install dependency "Adafruit Unified Sensor", klik Install All
- Library siap digunakan dengan #include <DHT.h>

e. Verifikasi Instalasi

Untuk memastikan semua library terinstall dengan benar:

- Buat sketch baru: File > New
- Coba include semua library:

```
#include <lvgl.h>
#include <TFT_eSPI.h>
#include <DHT.h>
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Serial.println("Libraries loaded successfully!");
}

void loop() {
  // Empty
}
```

- Klik Verify (ikon centang)
- Jika tidak ada error, semua library sudah terinstall dengan benar
- Jika ada error, periksa:
 - Versi library yang compatible

- Konfigurasi lv_conf.h dan User_Setup.h
- Path library yang correct

2. Instalasi dan Konfigurasi EEZ Studio

a. Download EEZ Studio

- 1) Buka browser dan akses GitHub repository EEZ Studio di <https://github.com/eez-open/studio/releases>
- 2) Scroll ke bagian "Assets" pada release terbaru (pilih versi stable, bukan pre-release)
- 3) Download installer sesuai sistem operasi:
 - Windows: download file ee-z-studio-setup-x.x.x.exe
 - macOS: download file ee-z-studio-x.x.x.dmg
 - Linux: download file ee-z-studio-x.x.x.AppImage
- 4) Ukuran file sekitar 100-200 MB, tunggu hingga download selesai

b. Instalasi EEZ Studio

Untuk Windows

- 1) Double-click file installer ee-z-studio-setup-x.x.x.exe
- 2) Jika muncul Windows SmartScreen warning, klik "More info" kemudian "Run anyway"
- 3) Ikuti installation wizard:
 - Accept license agreement
 - Pilih destination folder (default: C:\Program Files\EEZ Studio)
 - Pilih Start Menu folder
 - Centang "Create desktop shortcut" jika ingin shortcut di desktop
- 4) Klik Install dan tunggu proses instalasi selesai
- 5) Klik Finish untuk menyelesaikan instalasi

Untuk macOS

- 1) Double-click file ee-z-studio-x.x.x.dmg
- 2) Drag icon EEZ Studio ke folder Applications
- 3) Buka folder Applications dan double-click EEZ Studio
- 4) Jika muncul warning "EEZ Studio cannot be opened because the developer cannot be verified":
 - Buka System Preferences > Security & Privacy
 - Klik "Open Anyway" di bagian bawah
 - Confirm dengan klik "Open"

Untuk Linux

- 1) Beri permission executable pada file AppImage:

```
chmod +x ee-z-studio-x.x.x.AppImage
```

- 2) Jalankan EEZ Studio:

```
./ee-z-studio-x.x.x.AppImage
```

- 3) Opsional: install ke system untuk akses lebih mudah

- 4) Jika ada dependency missing, install sesuai error message (biasanya libfuse2)
- c. First Launch dan Initial Setup
 - 1) Jalankan EEZ Studio untuk pertama kali
 - 2) Pada welcome screen, pilih opsi:
 - Create New Project: untuk membuat project baru
 - Open Existing Project: untuk membuka project yang sudah ada
 - Recent Projects: list project yang pernah dibuka (kosong di first launch)
 - 3) Klik Create New Project
 - 4) Pilih project template:
 - Pilih "LVGL" sebagai project type
 - Beri nama project (misalnya: "WeatherStationUI")
 - Pilih lokasi penyimpanan project
 - 5) Klik Create untuk membuat project baru

1.4 Pembuatan Tampilan dan Elemen Interaktif di EEZ Studio

1.4.1 Perancangan User Interface

Tahap perancangan adalah proses perencanaan sebelum mulai mendesain di EEZ Studio.

1. Analisis Kebutuhan: Tujuannya adalah membuat satu layar (screen) untuk memantau data cuaca. Data yang akan ditampilkan adalah: Hujan, Suhu, Kelembaban, dan Cahaya.
2. Penentuan Properti Proyek: Proyek akan diatur sebagai berikut:
 - Project type: LVGL
 - LVGL version: 8.x
 - Display width: 320
 - Display height: 240
3. Sketsa Tata Letak (Layout):
 - Akan ada satu layar (screen) utama, yang kita beri nama "Main".
 - Di bagian atas layar, akan ada judul utama: **"DETEKSI CUACA"**.
 - Di bawah judul, akan ada grid 2x2 (dua kolom, dua baris) untuk menampung empat panel data.
 - Setiap panel akan berisi dua elemen label yang tersusun vertikal:
 - Label Judul (misalnya, "Suhu", "Hujan").
 - Label Data (misalnya, "0.0 °C", "...").

1.4.2 Membuat Layout dan Container Utama

1. **Buat Proyek Baru:** Buka EEZ Studio dan buat proyek baru (misalnya dengan nama "mencoba").
2. **Atur Properti Proyek:** Buka menu **Project -> Settings**. Di tab **General**, atur **Display width** ke **320** dan **Display height** ke **240**. Pastikan **LVGL version** adalah **8.x**.

3. **Buat Layar (Screen):** Di panel **Pages** (kiri atas), klik ikon tambah (+) untuk membuat layar baru. Di panel **Properties** (kanan), ubah **Name** menjadi **"Main"**.
4. **Atur Layout Layar "Main":**
 - Pilih screen **"Main"** di panel *Pages* atau *Widget Structure*.
 - Di panel *Properties* -> **Position and Size**, atur **Padding** (misalnya *Top, Bottom, Left, Right*) untuk memberi jarak dari tepi layar.
 - Di *Properties* -> **Layout**, atur **Display** ke **Flex**.
 - Atur **Flex Flow** ke **Column** (ini akan menyusun elemen-elemen di dalamnya secara vertikal).
5. **Tambahkan Container untuk Grid:**
 - Dari **Components Palette** (kanan bawah), seret (drag) widget **Container** ke dalam layar **"Main"**.
 - Pilih *Container* baru ini. Di *Properties* -> **Position and Size**, atur **Width** ke **100%** dan **Grow** ke **1**. Ini membuatnya mengisi sisa ruang di bawah judul.
 - Di *Properties* -> **Layout**, atur **Display** ke **Grid**.
 - Atur **Grid Column Descriptors** ke **1fr 1fr** (dua kolom dengan lebar sama).
 - Atur **Grid Row Descriptors** ke **1fr 1fr** (dua baris dengan tinggi sama).
 - Atur **Grid Column Gap** dan **Grid Row Gap** (misalnya 10px) untuk memberi jarak antar panel.

1.4.3 Menambahkan Elemen Dasar

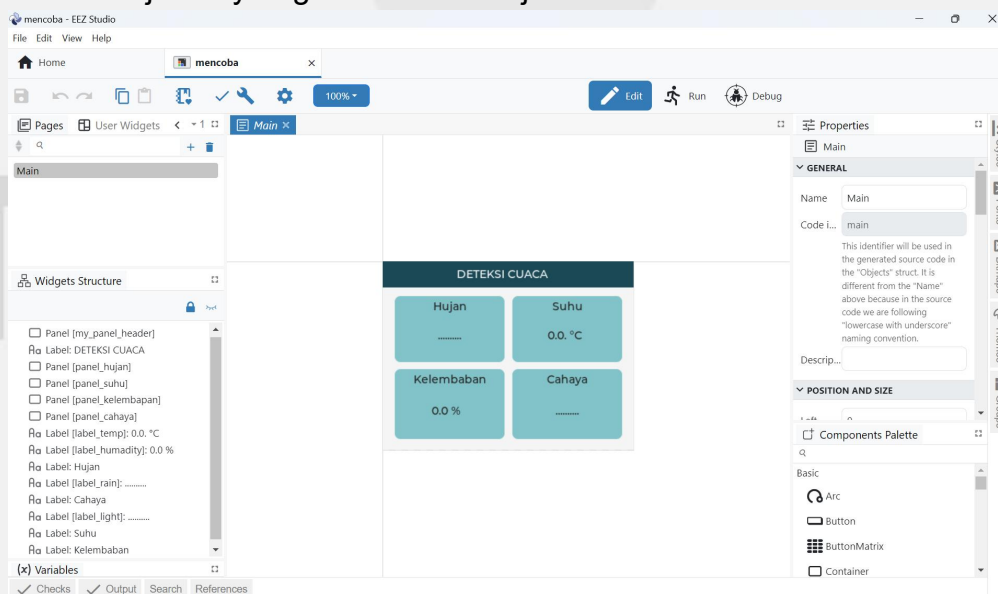
1. **Tambahkan Judul Utama:**
 - Seret widget **Label** dari *Components Palette* ke layar **"Main"**. *Pastikan label ini berada di atas container grid.*
 - Pilih *Label* tersebut. Di *Properties* -> **General**, ubah **Text** menjadi **"DETEKSI CUACA"**.
 - Di *Properties* -> **Text**, atur **Text Align** ke **Center**.
 - Kita juga bisa mengubah **Font** agar lebih besar dan tebal.
2. **Tambahkan Panel Data (4 buah):**
 - Seret 4 (empat) widget **Panel** (atau **Container**) dari *Components Palette* ke dalam **Container Grid** yang sudah buat di langkah 1.4.2. Panel-panel ini akan otomatis mengisi sel grid 2x2.
 - Untuk setiap panel, lakukan *styling*:
 - Pilih panel.
 - Di *Properties* -> **Style (Main)**, atur **Background Color** ke warna biru tosca atau sesuai selera.
 - Atur **Border Radius** (misalnya 10) agar sudutnya melengkung.
3. **Atur Layout Internal Panel:**
 - Pilih **setiap** panel (satu per satu).
 - Di *Properties* -> **Layout**, atur **Display** ke **Flex** dan **Flex Flow** ke **Column**.

- Atur **Justify Content** ke **Center** dan **Align Items** ke **Center**. Ini akan membuat label di dalam panel otomatis berada di tengah secara vertikal dan horizontal.

1.4.4 Menambahkan Widget Interaktif dan Indikator

Di sini kita menambahkan label yang akan menampilkan data (sebagai indikator).

1. **Identifikasi Panel:** Beri nama yang jelas untuk setiap panel di *Properties* -> **General** -> **Name** (sesuai *Widget Structure* di gambar): **panel_hujan**, **panel_suhu**, **panel_kelembaban**, **panel_cahaya**.
2. **Tambahkan Label ke Setiap Panel:** Untuk setiap panel, lakukan dua langkah berikut:
 - **Label Judul Panel:** Seret **Label** ke dalam panel. Ubah **Text**-nya (misalnya, "Hujan", "Suhu", "Kelembaban", "Cahaya"). Beri **Name** yang sesuai (misalnya **label_rain**, **label_suhu**).
 - **Label Data (Indikator):** Seret **Label** kedua ke dalam panel yang sama (di bawah label judul).
 - Untuk "Suhu", ubah **Text** ke "0.0 °C" dan beri **Name** **label_temp**.
 - Untuk "Kelembaban", ubah **Text** ke "0.0 %" dan beri **Name** **label_humidity**.
 - Untuk "Hujan" dan "Cahaya", ubah **Text** ke "..." dan beri nama (misalnya **label_data_hujan**, **label_data_cahaya**).
3. **Styling Label Data:** Pilih label data (misalnya **label_temp**). Di *Properties* -> **Text**, ubah **Font** agar ukurannya lebih besar dan mungkin **Color**-nya berbeda dari label judulnya agar lebih menonjol.



Gambar 1.5. Tampilan Design yang sudah dibuat

1.4.5 Export Code dari EEZ Studio

Setelah desain visual selesai, langkah terakhir adalah memverifikasi, meng-compile (Build), dan mengekspor desain tersebut menjadi kode LVGL yang siap pakai untuk *platform* target (seperti Arduino, ESP32, dll.).

a. Persiapan Export

Sebelum melakukan build dan export, ada baiknya melakukan verifikasi akhir untuk memastikan semua desain sudah final dan berfungsi.

1. Verifikasi Semua Elemen:

- Pastikan semua panel, label, dan *variables* sudah diberi nama dengan benar dan sesuai (misalnya, **label_temp**, **panel_suhu**).
- Jalankan simulator (tekan **F5** atau tombol **Debug/Run**) untuk melakukan pengujian terakhir pada tampilan.
- Periksa kembali panel "**Output**" atau "**Console**" dan pastikan tidak ada *error* atau *warning* yang muncul saat simulasi.

2. Pastikan Project Tersimpan:

- Tekan **Ctrl + S** atau pilih menu **File** → **Save**.
- Pastikan semua perubahan terakhir pada desain sudah tersimpan sebelum melanjutkan.

b. Build (Compile) Project

Proses *build* adalah proses *compile* desain visual menjadi file kode C (.c) dan header (.h).

1. Mengakses Menu Build:

- Klik menu **Project** → **Build** di *menubar* atas.
- Atau, klik ikon **Build** (tang) di *toolbar*.
- Atau, tekan *shortcut* **Ctrl + B** (jika tersedia).

2. Proses Build:

- EEZ Studio akan mulai meng-compile proyek.
- Kita dapat memantau progresnya di panel "**Output**" (biasanya di bagian bawah).
- Tunggu hingga proses selesai dan muncul pesan sukses, seperti **Build finished successfully** atau **Build completed: 0 errors, 0 warnings**

c. Jika Ada Error:

- Baca dengan teliti pesan *error* yang muncul di panel **Output**.
- Pesan tersebut akan menunjukkan masalahnya (misalnya, nama variabel yang salah, *syntax* yang tidak valid, atau properti yang hilang).
- Perbaiki *error* tersebut di dalam editor visual.
- Lakukan proses **Build** ulang.

1.5 Implementasi Code di Arduino IDE

1.5.1 Import Generated Code ke Arduino IDE

1. Instalasi Library LVGL (dari Git/Folder):perangkat IoT

- Jika kita telah mengunduh LVGL sebagai folder (misalnya, dari Git), tidak bisa menambahkannya melalui Library Manager biasa.
- Di Arduino IDE, buka menu **Sketch** → **Include Library** → **Add .ZIP Library....**

- Pilih folder **.zip** dari *library* LVGL yang telah diunduh.
- Jika sudah memilikinya sebagai folder yang sudah di-unzip, bisa menyalin folder tersebut secara manual ke direktori **...\Documents\Arduino\libraries**.

2. Konfigurasi **lv_conf.h**:

- Buka folder *library* LVGL yang baru saja instal (misal, **...\Documents\Arduino\libraries\lvgl**).
- Cari file bernama **lv_conf_template.h**.
- Salin (copy) dan tempel (paste) file ini di lokasi yang sama, lalu ganti namanya (rename) menjadi **lv_conf.h**.
- **Penting:** Buka file **lv_conf.h** tersebut. Di bagian paling atas, ubah **#if 0** menjadi **#if 1**. Ini akan mengaktifkan konfigurasi di dalam file tersebut.
- Pastikan juga mengatur resolusi di **lv_conf.h** agar sesuai dengan proyek kita. (misal: **LV_HOR_RES_MAX** ke 320 dan **LV_VER_RES_MAX** ke 240) dan mengaktifkan *font* atau *widget* yang mungkin butuhkan.

3. Cari Folder Sketch untuk Kode UI:

- Buat *sketch* baru dan simpan (misal, **DeteksiCuaca**).
- Buka folder *sketch* (**Sketch → Show Sketch Folder**).
- Strukturnya: **...\DeteksiCuaca\DeteksiCuaca.ino**

4. Salin File UI yang Diekspor:

- Salin (copy) **semua file .c dan .h** yang telah di ekspor dari EEZ Studio ke dalam folder Sketch yang baru saja dibuat.

1.5.2 Inisialisasi Display dan LVGL

Bagian ini berisi kode *boilerplate* yang diperlukan di dalam fungsi **setup()** untuk mengaktifkan display, LVGL, dan memuat UI yang sudah di desain. LVGL perlu dipanggil secara rutin untuk menangani *rendering* (menggambar ulang layar) dan memproses input. Ini harus dilakukan di dalam **loop()** utama.

1.5.2.1 Bagian Include dan Definisi Global

```
#include <lvgl.h>
#include <TFT_eSPI.h>
#include "ui.h" // hasil export dari EEZ Studio atau SquareLine Studio

// [LEVEL 1] ----- KONFIGURASI DISPLAY -----
#define SCREEN_WIDTH 320
#define SCREEN_HEIGHT 240
#define DRAW_BUF_SIZE (SCREEN_WIDTH * SCREEN_HEIGHT / 10 * (LV_COLOR_DEPTH / 8))
```

Penjelasan Kode:

- **#include <lvgl.h>**: Memasukkan *library* inti LVGL. Ini menyediakan semua fungsi untuk membuat *widget* (tombol, label, slider, dll).
- **#include <TFT_eSPI.h>**: Memasukkan *library* driver untuk layar TFT Anda. LVGL sendiri tidak tahu cara berbicara langsung ke perangkat keras layar; ia membutuhkan *library* ini sebagai perantara.
- **#include "ui.h"**: Ini adalah bagian penting yang Anda dapatkan dari *software* desain UI. File ini berisi fungsi `ui_init()` yang "membangun" semua layar dan *widget* yang telah Anda desain secara visual.
- **SCREEN_WIDTH & SCREEN_HEIGHT**: Mendefinisikan resolusi layar Anda.
- **DRAW_BUF_SIZE**: Ini mendefinisikan seberapa besar *memory* (RAM) yang Anda alokasikan untuk *buffer* gambar. LVGL tidak menggambar piksel satu per satu langsung ke layar. Sebaliknya, ia menggambar ke *buffer* internal terlebih dahulu.
 - Rumus $(SCREEN_WIDTH * SCREEN_HEIGHT / 10 \dots)$ adalah optimasi umum. Ini mengalokasikan *buffer* yang besarnya **1/10 (sepersepuluh)** dari total ukuran layar. Ini menghemat banyak RAM, terutama pada mikrokontroler yang terbatas. LVGL akan menggambar layar dalam 10 "potongan" (chunks).

1.5.2.2 Bagian Objek Global

```
TFT_eSPI tft = TFT_eSPI();
lv_disp_draw_buf_t draw_buf;
lv_color_t buf1[DRAW_BUF_SIZE / sizeof(lv_color_t)];
uint32_t lastTick = 0;
```

Penjelasan Kode:

- **TFT_eSPI tft = TFT_eSPI();**: Membuat sebuah "objek" dari *library* TFT_eSPI. Objek `tft` inilah yang akan kita gunakan untuk semua perintah ke layar (seperti inisialisasi, mengatur area gambar, dan mengirim piksel).
- **lv_disp_draw_buf_t draw_buf;**: Mendeklarasikan sebuah struktur data LVGL yang akan menyimpan informasi tentang *buffer* gambar kita.
- **lv_color_t buf1[...];**: Ini adalah **alokasi memori yang sebenarnya** untuk *buffer* gambar. `buf1` adalah *array* di RAM tempat LVGL akan menggambar. Ukurannya kita ambil dari `DRAW_BUF_SIZE` yang sudah kita hitung sebelumnya.

1.5.2.3 Callback (Fungsi Jembatan) untuk Display

```
void my_disp_flush(lv_disp_drv_t *disp, const lv_area_t *area,
lv_color_t *color_p) {
    uint32_t w = (area->x2 - area->x1 + 1);
    uint32_t h = (area->y2 - area->y1 + 1);
```

```

tft.startWrite();
tft.setAddrWindow(area->x1, area->y1, w, h);
tft.pushColors((uint16_t *)&color_p->full, w * h, true);
tft.endWrite();

lv_disp_flush_ready(displ);
}

```

Penjelasan Kode:

Ini adalah bagian **paling penting** yang menghubungkan LVGL dan TFT_eSPI.

- **Apa itu "Callback"?** Ini adalah fungsi yang kita tulis, tetapi kita tidak pernah memanggilnya secara langsung. Sebaliknya, kita memberitahu LVGL, "Hei LVGL, setiap kali kamu selesai menggambar sesuatu di *buffer*, tolong panggil fungsi **my_disp_flush** ini ya."
- **Apa yang dilakukannya?**
 1. LVGL memanggil fungsi ini dan memberikan tiga informasi: **disp** (display mana), **area** (bagian layar mana yang baru saja digambar), dan **color_p** (pointer ke *buffer* yang berisi data piksel baru).
 2. **w** dan **h**: Kita menghitung lebar dan tinggi dari **area** yang perlu diperbarui.
 3. **tft.startWrite()**: Memberi tahu layar TFT untuk bersiap menerima data.
 4. **tft.setAddrWindow(...)**: Memberi tahu layar, "Saya hanya akan mengirim data untuk area kotak dari (x1, y1) ke (x2, y2)." Ini adalah optimasi besar sehingga kita tidak perlu me-refresh seluruh layar setiap saat.
 5. **tft.pushColors(...)**: Perintah ini **mengirimkan data piksel** dari *buffer* LVGL (**color_p**) ke layar TFT.
 6. **tft.endWrite()**: Selesai mengirim data.
 7. **lv_disp_flush_ready(displ);**: Ini adalah sinyal kita kembali ke LVGL, "OK, saya sudah selesai mengirim *buffer* tadi ke layar. Anda boleh lanjut menggambar bagian berikutnya."

1.5.2.4 Fungsi Setup (Inisialisasi)

```

void setup() {
  Serial.begin(115200);
  Serial.println("Starting LVGL (Level 1)...");

  // Inisialisasi layar
  tft.begin();
  tft.setRotation(1);
}

```



```

// Inisialisasi LVGL
lv_init();

// Buffer gambar LVGL
lv_disp_draw_buf_init(&draw_buf, buf1, NULL, SCREEN_WIDTH * 10);

// Driver display LVGL
static lv_disp_drv_t disp_drv;
lv_disp_drv_init(&disp_drv);
disp_drv.hor_res = SCREEN_WIDTH;
disp_drv.ver_res = SCREEN_HEIGHT;
disp_drv.flush_cb = my_disp_flush;
disp_drv.draw_buf = &draw_buf;
lv_disp_drv_register(&disp_drv);

// Inisialisasi UI dari EEZ/SquareLine
ui_init();

Serial.println("LVGL initialized and UI loaded!");
}

```

Penjelasan Kode:

Fungsi **setup()** dieksekusi sekali saat mikrokontroler menyala.

1. **Serial.begin(115200)**: Memulai komunikasi Serial untuk *debugging*.
2. **tft.begin()**: Menginisialisasi *library* TFT_eSPI.
3. **tft.setRotation(1)**: Mengatur rotasi layar. (Angka **1** biasanya berarti *landscape*/horizontal).
4. **lv_init()**: Menginisialisasi *library* LVGL.
5. **lv_disp_draw_buf_init(...)**: Memberitahu LVGL untuk menggunakan **buf1** sebagai memori *buffer*-nya. (**NULL** berarti kita hanya pakai *single buffer*).
6. **Blok Driver Display**:
 - **lv_disp_drv_init(&disp_drv)**: Mempersiapkan struktur driver display.
 - **disp_drv.hor_res = ...**: Memberitahu LVGL ukuran resolusi layar.
 - **disp_drv.ver_res = ...**: Memberitahu LVGL ukuran resolusi layar.
 - **disp_drv.flush_cb = my_disp_flush**: Ini **kuncinya!** Kita memberitahu LVGL untuk menggunakan fungsi **my_disp_flush** kita sebagai *callback* pengiriman data.
 - **disp_drv.draw_buf = &draw_buf**: Memberitahu LVGL untuk menggunakan konfigurasi *buffer* yang sudah kita siapkan.

- `lv_disp_drv_register(&disp_drv);`: Mendaftarkan driver ini ke LVGL agar aktif.
7. `ui_init()`: Ini adalah fungsi dari file `ui.h`. Memanggil fungsi ini akan **secara otomatis menggambar semua layar, tombol, dan label** yang telah Anda buat di SquareLine Studio ke layar.

1.5.2.5 Fungsi Loop (Jantung Program)

```
void loop() {  
    lv_timer_handler(); // Jalankan LVGL  
    delay(3);           // Jeda kecil agar LVGL tetap responsif  
}
```

Penjelasan Kode:

Fungsi `loop()` berjalan berulang-ulang selamanya.

- `lv_timer_handler()`: Ini adalah "jantung" dari LVGL. Anda **wajib** memanggil fungsi ini sesering mungkin. Fungsi inilah yang menangani semua tugas LVGL, seperti:
 - Menjalankan animasi (misalnya, tombol berpindah tempat).
 - Mengecek apakah ada bagian layar yang perlu digambar ulang.
 - Memproses *event* (jika ada input).
- `delay(3)`: Ini adalah jeda singkat (3 milidetik) untuk memberi "napas" pada prosesor. Tanpa jeda ini, `loop` akan berjalan terlalu cepat dan bisa memonopoli prosesor. Angka 3-5ms adalah nilai yang umum dan aman.

1.6 Pengujian dan Debug Tampilan

1.6.1 Testing di EEZ Studio

Selalu lakukan pengujian di dalam simulator EEZ Studio terlebih dahulu. Ini adalah cara tercepat untuk memverifikasi desain visual.

1. **Jalankan Simulator:** Tekan tombol **Debug/Run** (ikon 'Play' ►) atau tekan **F5**.
2. **Verifikasi Tampilan:** Sebuah jendela simulasi akan muncul. Periksa hal-hal berikut:
 - **Layout:** Apakah semua panel (Hujan, Suhu, dll.) berada di posisi 2x2 yang benar?
 - **Teks:** Apakah judul "DETEKSI CUACA" dan semua label panel terlihat jelas dan tidak terpotong?
 - **Ukuran:** Pastikan tampilan di simulator sesuai dengan resolusi target (320x240).
3. **Keuntungan:** Proses ini sangat cepat, bisa langsung melihat perubahan visual tanpa harus menunggu proses *compile* dan *upload* Arduino yang lama.

1.6.2 Upload dan Testing di Hardware

Ini adalah pengujian sebenarnya pada perangkat mikrokontroler dan display TFT.

1. **Konfigurasi Arduino IDE:**

- Pastikan kita telah memilih **Board** yang benar (misal, "ESP32 Dev Module").
 - Pastikan kita telah memilih **Port** COM yang sesuai.
2. **Upload Kode:** Klik tombol **Upload** (ikon panah →) di Arduino IDE dan tunggu prosesnya selesai.
3. **Observasi Perangkat Keras:**
- **Layar Menyala?** Apakah layar TFT menyala dan menampilkan UI?
 - **Visual Sesuai?** Bandingkan tampilan di layar fisik dengan simulator EEZ Studio. Apakah warna, font, dan posisi elemen sudah sama?
 - **Data Update?** Perhatikan label data (Suhu dan Kelembaban). Berkat kode di `loop()` pada 1.5.6, seharusnya melihat nilai *dummy* (acak) berubah setiap detik. Ini mengonfirmasi bahwa *task handler* LVGL dan *logic update* berfungsi.

1.6.3 Debugging dengan Serial Monitor Arduino

Jika tampilan tidak muncul, data tidak *update*, atau perangkat *crash*, Serial Monitor adalah alat *debug* terbaik.

1. **Buka Serial Monitor:** Di Arduino IDE, klik ikon **Serial Monitor** (kaca pembesar). Pastikan *baud rate* diatur ke **115200** (sesuai `Serial.begin(115200);` di `setup()`).
2. **Tambahkan `Serial.println()`:** Letakkan "checkpoint" di dalam kode untuk melacak progres.
 - **Di `setup()`:** Tambahkan `Serial.println()` setelah setiap langkah inisialisasi penting untuk menemukan di mana program berhenti jika terjadi *crash*.

```
void setup() {
    Serial.begin(115200);
    Serial.println("Booting...");

    lv_init();
    Serial.println("LVGL init OK");

    tft.begin();
    tft.setRotation(1);
    Serial.println("TFT init OK");

    lv_disp_drv_register(&disp_drv);
    Serial.println("Display driver registered OK");

    ui_init(); // Fungsi dari EEZ Studio
    Serial.println("UI init OK. Setup complete.");
}
```

- **Di `loop()`:** Cetak nilai *dummy* sebelum menampilkannya ke UI. Ini untuk memastikan bahwa *logic* pembacaan data berjalan, meskipun UI-nya belum ter-update.

```
if (millis() - lastDataUpdate >= 1000) {
    float dummyTemp = random(250, 300) / 10.0;
    Serial.print("Updating data. Temp: "); // DEBUG
    Serial.println(dummyTemp);           // DEBUG

    lv_label_set_text_fmt(ui_label_temp, "%.1f °C",
dummyTemp);
    // ...
    lastDataUpdate = millis();
}
```

1.6.4 Common Issues dan Solusinya

- a. **Masalah:** Layar putih (White Screen).

Solusi: Ini hampir selalu masalah *driver* display.

1. Periksa kembali konfigurasi pin di `User_Setup.h` (jika menggunakan TFT_eSPI).
2. Periksa koneksi/solderan pin CS, DC, RST, SCLK, dan MOSI.
3. Pastikan `tft.begin()` dipanggil di `setup()`.

- b. **Masalah:** Tampilan "berantakan", terpotong, atau artefak grafis.

Solusi: Resolusi tidak cocok.

1. Pastikan resolusi di EEZ Studio (**320x240**) SAMA PERSIS dengan `SCREEN_WIDTH / SCREEN_HEIGHT` di kode Arduino dan `lv_conf.h`.

- c. **Masalah:** UI Tampil tapi data (angka) tidak pernah berubah (UI "beku").

Solusi: *Handler* LVGL tidak berjalan atau *loop* terblokir.

1. Pastikan `lv_timer_handler()` dipanggil di setiap `loop()`.
PENTING: Pastikan **tidak menambahkan `delay()` yang lama** (misal, `delay(1000)`) di dalam `loop()`. *Delay* singkat seperti `delay(3)` atau `delay(5)` (seperti di kode sebelumnya) tidak masalah, tetapi *delay* yang panjang akan memblokir *update* data `millis()` dan membuat UI tidak responsif.

- d. **Masalah:** Error kompilasi "ui_label_temp was not declared in this scope".

Solusi: Kode UI tidak ter-include atau nama *widget* salah.

1. Pastikan kita memiliki `#include "ui.h"` (atau `#include "ui/ui.h"`) di atas file `.ino`.
2. Buka EEZ Studio, klik label suhu, dan periksa *Properties* -> **Name**. Nama ini (misal, `label_temp`) harus diawali dengan `ui_` (menjadi `ui_label_temp`) saat dipanggil di kode Arduino.

1.6.5 Optimasi Performa

Jika UI terasa lambat atau berkedip (*flickering*):

1. **Sesuaikan Ukuran Buffer:** Di file `.ino`, `DRAW_BUF_SIZE`.
 - Menggunakan *buffer* lebih besar (misal, ubah `/ 10` menjadi `/ 8` atau `/ 5`) akan menggunakan lebih banyak RAM tetapi bisa membuat *refresh* layar lebih cepat.
 - Menggunakan *buffer* lebih kecil (misal, `/ 20`) menghemat RAM tetapi bisa membuat *rendering* lebih lambat.
2. **Atur SPI Clock Speed:** (Khusus TFT_eSPI) Di file `User_Setup.h`, coba naikkan `SPI_FREQUENCY` (misal, ke `40000000` atau `80000000`) untuk transfer data ke display yang lebih cepat.
3. **Batasi Update UI:** Jangan panggil `lv_label_set_text()` di setiap putaran `loop()`. Gunakan *timer* `millis()` (seperti yang sudah kita lakukan di 1.5.6) untuk me-refresh data hanya saat diperlukan (misal, setiap 1 detik).

1.7 Kesimpulan

Chapter 1 ini membahas dasar-dasar IoT untuk membuat stasiun cuaca mini menggunakan ESP32 dan LVGL. Materi mencakup pengenalan tiga software utama: LVGL sebagai library grafis yang ringan, EEZ Studio untuk desain UI secara visual, dan Arduino IDE untuk programming dan upload ke ESP32.

Pembahasan meliputi persiapan hardware (TFT LCD Display dan kabel USB), instalasi software, pembuatan tampilan dashboard 2x2 untuk data cuaca, serta implementasi kode untuk menampilkan data secara real-time. Bagian pengujian menjelaskan cara testing dari simulator hingga hardware, troubleshooting masalah umum, dan optimasi performa tampilan.

Melalui chapter ini, peserta memperoleh pengetahuan dan keterampilan dasar untuk mengembangkan sistem IoT dengan antarmuka yang profesional dan user-friendly menggunakan kombinasi LVGL, EEZ Studio, dan Arduino IDE.

CHAPTER 2

Membangun Stasiun Cuaca Mini dan Mengirim Data ke LVGL

2.1 Pendahuluan

2.1.1 Latar Belakang

Perubahan iklim dan ketidakstabilan kondisi cuaca merupakan salah satu dampak dalam kehidupan sehari-hari, seperti kasus pada beberapa sektor seperti pertanian, perkotaan, lingkungan terbuka seperti taman. Dalam kondisi saat ini suhu yang tidak menentu, perubahan datangnya hujan, dan meningkatnya cuaca ekstrem telah menimbulkan tantangan dalam pengelolaan sumber daya dan perencanaan aktivitas Informasi mengenai kondisi cuaca seperti suhu, kelembaban udara, dan curah hujan sangat diperlukan untuk mendukung aktivitas sehari-hari, mulai dari perencanaan kegiatan luar ruangan hingga pengelolaan infrastruktur publik.

Dalam permasalahan tersebut, Sistem pemantauan cuaca yang mampu memberikan informasi secara real-time, akurat, dan dapat diakses dengan mudah diperlukan untuk saat ini. Teknologi Internet of Things (IoT) memberikan solusi efektif dengan membuat stasiun mini cuaca dimana ini menghubungkan berbagai sensor yang dapat secara otomatis mengumpulkan dan mengirimkan data lingkungan ke platform digital. Melalui penerapan IoT, proses pengamatan cuaca tidak lagi bergantung pada pengukuran manual, tetapi dapat dilakukan secara berkelanjutan dan terintegrasi (Kumar et al., 2024).

Namun, stasiun cuaca konvensional umumnya memiliki keterbatasan dalam hal aksesibilitas, biaya pengadaan yang tinggi, dan jangkauan monitoring yang terbatas pada lokasi-lokasi tertentu. Menurut Al Kalaany et al. (2025), sistem stasiun cuaca berbasis IoT berbiaya rendah mampu memberikan data mikroklimat yang berguna untuk berbagai aplikasi, seperti peringatan dini bencana, pemantauan kualitas udara, serta analisis perubahan lingkungan.

Dalam beberapa Penelitian sistem stasiun cuaca kecil berbasis IoT untuk pemantauan cuaca pada lingkungan telah terbukti secara akurat. Botero-Valencia et al. (2022) mengembangkan stasiun mini cuaca yang dengan biaya yang rendah, alat ini mampu memantau suhu, kelembaban, dan curah hujan secara real-time untuk optimasi kegiatan pertanian. Penelitian Ganesan et al. (2024) menyoroti fleksibilitas sensor seperti DHT11, dan sensor hujan ke dalam arsitektur IoT yang mudah dikembangkan sesuai kebutuhan pengguna.

Selain untuk pemantauan umum, beberapa studi kasus juga menunjukkan penerapan stasiun cuaca IoT dalam bidang pertanian spesifik. Rifka et al. (2024) misalnya, mengimplementasikan sistem pemantauan curah hujan berbasis IoT pada lahan pertanian cabai untuk membantu petani menentukan jadwal penyiraman yang efisien. Penelitian Faid et al. (2021) bahkan menggabungkan sistem IoT dengan kecerdasan buatan (AI) untuk meningkatkan analisis prediktif dan otomatisasi dalam pengambilan keputusan pertanian.

Berdasarkan hasil penelitian tersebut, pengembangan sistem stasiun cuaca mini berbasis IoT sangat relevan. Selain memberikan solusi pemantauan lingkungan yang

ekonomis, sistem ini juga membuka peluang bagi penerapan teknologi digital di sektor pertanian dan pendidikan. Modul ini disusun dengan tujuan untuk memberikan pemahaman konseptual dan praktis tentang cara merancang, membangun, serta mengimplementasikan sistem stasiun cuaca mini berbasis IoT. Melalui modul ini, diharapkan pengguna dapat memahami keterkaitan antara teknologi sensor, komunikasi data, dan analisis cuaca real-time sebagai bagian dari upaya menuju sistem pemantauan lingkungan yang cerdas, efisien, dan berkelanjutan.

2.1.2 Tujuan Pembelajaran

Setelah mempelajari modul ini, peserta didik diharapkan mampu :

1. Memahami prinsip kerja sistem stasiun cuaca mini serta fungsi utama komponen sensor yang digunakan, seperti sensor suhu dan kelembaban (DHT11) dan sensor hujan.
2. Merancang dan merakit rangkaian sensor sistem sensor dengan menggunakan mikrokontroler LVGL ESP32-2432S032R_I sebagai pusat pengendali dan pengolahan data.
3. Mengonfigurasi dan memprogram LVGL ESP32-2432S032R_I untuk membaca data dari berbagai sensor secara real-time menggunakan bahasa pemrograman berbasis Arduino IDE.
4. Menampilkan data hasil pengukuran dalam bentuk antarmuka grafis interaktif menggunakan pustaka LVGL agar informasi cuaca dapat diakses dengan mudah dan menarik.
5. Menganalisis hasil pengukuran dan kestabilan sistem, serta mengidentifikasi faktor-faktor yang mempengaruhi akurasi pembacaan sensor.

2.2 Konsep dan Cara Kerja Stasiun Cuaca Mini

2.2.1 Konsep Dasar Sistem IoT pada Stasiun Cuaca Mini

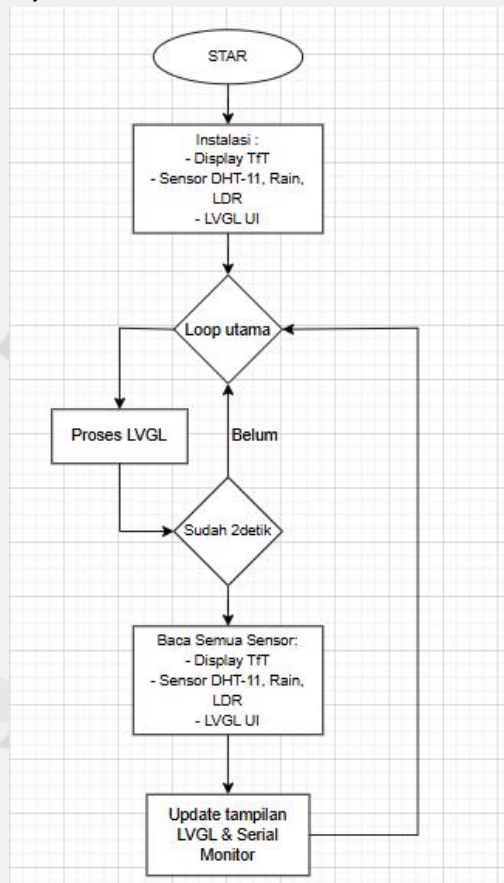
Stasiun cuaca mini berbasis IoT merupakan sistem terintegrasi yang menggabungkan tiga komponen utama: sensor sebagai pengumpul data, mikrokontroler sebagai pemroses informasi, dan antarmuka pengguna sebagai media visualisasi. Konsep dasar sistem ini merupakan prinsip Internet of Things dimana perangkat fisik (sensor) terhubung dengan sistem komputasi untuk menghasilkan informasi yang dapat diakses dan dimonitor secara real-time.

Dalam implementasinya, sistem stasiun cuaca mini bekerja melalui alur yang sistematis sensor-sensor akan menangkap parameter lingkungan seperti suhu, kelembaban, intensitas cahaya, dan keberadaan hujan, kemudian data tersebut dikirimkan ke mikrokontroler ESP32 yang bertindak sebagai pusat pengolahan data. ESP32 membaca, mengkonversi, dan memproses sinyal dari sensor menjadi informasi yang terstruktur. Selanjutnya, data yang telah diproses ditampilkan pada antarmuka grafis menggunakan library LVGL yang menyediakan visualisasi intuitif dan responsif pada layar TFT.

Keunggulan pendekatan IoT pada sistem ini terletak pada fleksibilitas dan skalabilitasnya. ESP32 yang dilengkapi dengan modul WiFi dan Bluetooth memungkinkan sistem untuk dikembangkan lebih lanjut dengan fitur-fitur seperti data

logging, remote monitoring melalui cloud platform, atau integrasi dengan sistem smart home dan smart agriculture.

2.2.2 Alur Data (Data Flow) Sistem



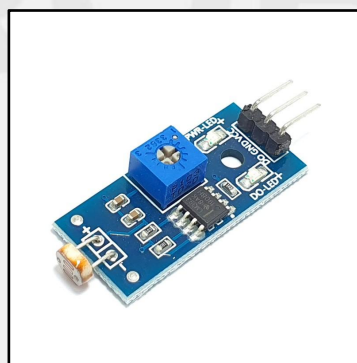
Gambar 2.1. Data Flow Sistem

2.3 Perakitan Perangkat dan Integrasi Sensor

2.3.1 Kebutuhan Perangkat

2.3.1.1 Perangkat Keras (Hardware)

a. Sensor LDR



Gambar 2.2. Sensor LDR

Sumber: [Sensor LDR](#)

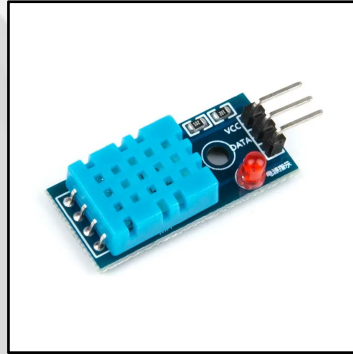
Sensor cahaya atau LDR (Light Dependent Resistor) digunakan untuk mendeteksi intensitas cahaya di sekitar. Resistansi sensor akan menurun saat menerima cahaya yang lebih terang, dan meningkat ketika berada di tempat gelap. Modul LDR sudah dilengkapi IC komparator (LM393) dan potensiometer

biru untuk mengatur tingkat sensitivitas pembacaan cahaya.

Sensor ini memiliki empat pin, yaitu:

- A0, merupakan output analog yang menghasilkan nilai tegangan sesuai intensitas cahaya.
- D0, merupakan output digital yang memberikan sinyal HIGH atau LOW tergantung pada batas cahaya yang diatur menggunakan potensiometer.
- GND, digunakan sebagai jalur ground (negatif).
- VCC, digunakan untuk memberikan tegangan kerja pada sensor (3,3V–5V).

b. Sensor DHT11



Gambar 2.3. Sensor DHT11

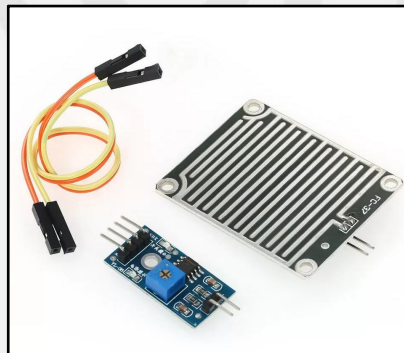
Sumber: [Sensor DHT11](#)

Sensor DHT11 berfungsi untuk mengukur suhu dan kelembaban udara. Sensor ini bekerja secara digital, di mana data suhu dan kelembaban dikirim melalui satu jalur data menuju mikrokontroler seperti ESP32. Sensor ini mudah digunakan karena sudah memiliki komponen pendukung di dalam modulnya.

Sensor DHT11 memiliki tiga pin utama, yaitu:

- VCC, berfungsi sebagai sumber tegangan untuk menghidupkan sensor (dapat menggunakan 3,3V atau 5V).
- DATA, digunakan untuk mengirimkan data hasil pembacaan suhu dan kelembaban ke mikrokontroler.
- GND, berfungsi sebagai jalur ground atau negatif dari rangkaian.

c. Sensor Raindrop



Gambar 2.4. Sensor Raindrop

Sumber: [Sensor Raindrop](#)

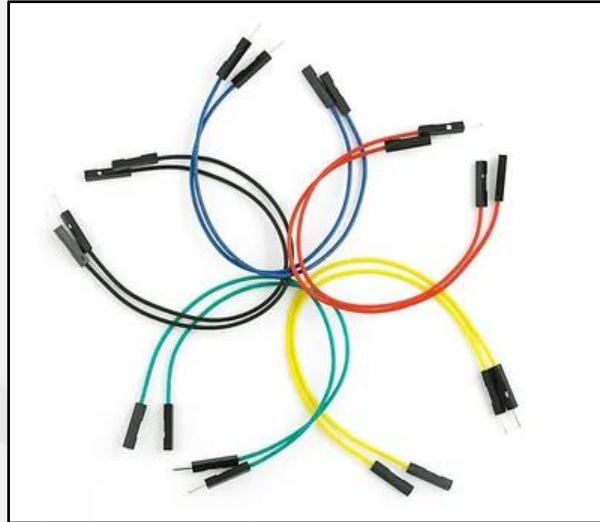
Sensor raindrop digunakan untuk mendeteksi adanya air hujan di lingkungan sekitar. Modul ini terdiri dari dua bagian, yaitu papan sensor (probe) yang mendeteksi tetesan air, dan papan kontrol yang berisi IC LM393 serta potensiometer untuk mengatur sensitivitas deteksi. Ketika air mengenai papan

sensor, nilai tegangan yang dihasilkan akan berubah seperti semakin banyak air, maka semakin rendah nilai tegangan.

Sensor raindrop memiliki empat pin utama, yaitu:

- A0, berfungsi sebagai output analog untuk menunjukkan tingkat basah atau jumlah air yang terdeteksi.
- D0, berfungsi sebagai output digital yang memberikan sinyal HIGH atau LOW berdasarkan batas deteksi air.
- GND, digunakan sebagai jalur ground (negatif).
- VCC, sebagai sumber tegangan (3,3V–5V) untuk menghidupkan sensor.

d. Kabel Jumper



Gambar 2.5. Kabel Jumper

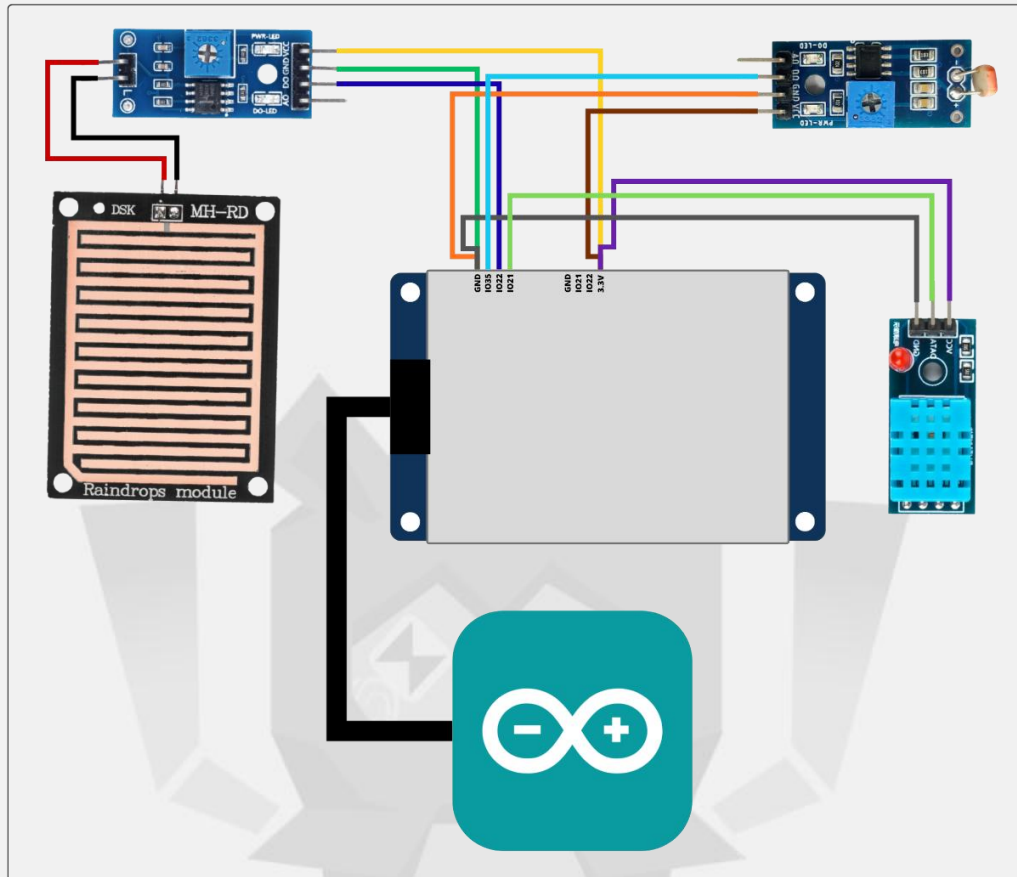
Sumber: [Kabel Jumper](#)

Kabel jumper berfungsi sebagai penghubung antar komponen elektronik di breadboard atau antara sensor dengan mikrokontroler. Terdapat tiga jenis utama: male-to-male, male-to-female, dan female-to-female. Kabel ini memudahkan proses perakitan, pengujian, dan modifikasi rangkaian tanpa perlu menyolder.

2.3.1.2 Perangkat Lunak (Software)

- a. Arduino IDE
- b. EEZ Studio

2.3.2 Skematik dan Rangkaian Perangkat



Gambar 2.6. Rangkaian Perangkat Stasiun Cuaca Mini

2.3.3 Proses Uji Koneksi Awal

a. Uji Sensor LDR

Pengujian ini dilakukan untuk mengidentifikasi apakah sensor LDR dapat mendeteksi perbedaan intensitas cahaya (terang/gelap) dengan benar.

```
#define LDR_PIN 35 // Pin digital yang digunakan untuk sensor LDR

void setup() {
  Serial.begin(115200);
  pinMode(LDR_PIN, INPUT);
  Serial.println("Uji Sensor LDR Dimulai...");
}

void loop() {
  int ldrValue = digitalRead(LDR_PIN);
  const char *lightStatus = (ldrValue == LOW) ? "Terang" : "Gelap";

  Serial.print("Nilai LDR: ");
  Serial.print(ldrValue);
  Serial.print(" | Kondisi: ");
```

```

Serial.println(lightStatus);

delay(1000);
}

```

Penjelasan:

- Kode ini membaca output digital dari sensor LDR.
- Jika kondisi terang, nilai LDR biasanya LOW (0), dan jika gelap, nilainya HIGH (1).
- Hasilnya akan tampil di Serial Monitor setiap 1 detik.

b. Uji Sensor DHT11

Pengujian ini dilakukan untuk menguji apakah sensor DHT11 dapat membaca suhu dan kelembaban udara dengan benar.

```

#include "DHT.h"

#define DHTPIN 21      // Pin data sensor
#define DHTTYPE DHT11 // Jenis sensor

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
  Serial.println("Uji Sensor DHT11 Dimulai...");
}

void loop() {
  float suhu = dht.readTemperature();
  float kelembapan = dht.readHumidity();

  if (isnan(suhu) || isnan(kelembapan)) {
    Serial.println("Gagal membaca data dari DHT11!");
  } else {
    Serial.print("Suhu: ");
    Serial.print(suhu);
    Serial.print(" °C | Kelembapan: ");
    Serial.print(kelembapan);
    Serial.println(" %");
  }

  delay(2000);
}

```

Penjelasan:

- Sensor DHT11 mengirimkan data suhu dan kelembaban secara digital melalui satu pin.
- Hasil pembacaan ditampilkan di Serial Monitor setiap 2 detik.

- Jika muncul pesan “Gagal membaca data”, berarti sensor belum tersambung atau belum terdeteksi dengan baik.

c. Uji Sensor Raindrop

Pengujian ini dilakukan untuk mengetahui apakah sensor raindrop dapat mendeteksi keberadaan air pada permukaan papan sensornya.

```
#define RAIN_PIN 22 // Pin digital D0 sensor hujan

void setup() {
  Serial.begin(115200);
  pinMode(RAIN_PIN, INPUT);
  Serial.println("Uji Sensor Hujan Dimulai...");
}

void loop() {
  int rainValue = digitalRead(RAIN_PIN);
  const char *rainStatus = (rainValue == LOW) ? "Hujan
  Terdeteksi" : "Tidak Hujan";

  Serial.print("Nilai Sensor Hujan: ");
  Serial.print(rainValue);
  Serial.print(" | Status: ");
  Serial.println(rainStatus);

  delay(1000);
}
```

Penjelasan:

- Sensor raindrop memberikan output **LOW (0)** saat ada air di permukaan sensor, dan **HIGH (1)** saat kering.
- Hasilnya ditampilkan di Serial Monitor setiap 1 detik untuk memastikan sensor berfungsi.

2.4 Pemrograman Pembacaan Data Sensor

2.4.1 Tambahan Include & Konfigurasi Pin

Lokasi:

- `#include "DHT.h"`: Taruh di bagian `include` paling atas.
- Blok "KONFIGURASI SENSOR": Taruh setelah blok "KONFIGURASI DISPLAY" Level 1.

```
#include "DHT.h" // [LEVEL 2] Library DHT11

// ----- KONFIGURASI DISPLAY -----

#define DHTPIN 21
#define DHTTYPE DHT11
#define RAIN_PIN 22
```

```
#define LDR_PIN 35
```

Penjelasan Kode:

- `#include "DHT.h"`: Mengimpor *library* Adafruit DHT. Ini memberi kita perintah-perintah khusus seperti `dht.readTemperature()`.
- `#define DHTPIN 21`: Membuat konstanta bernama `DHTPIN` yang bernilai `21`. Jadi, jika kita ingin memindah sensor ke pin lain, kita hanya perlu mengubah angka `21` di sini.
- `#define DHTTYPE DHT11`: Memberi tahu *library* DHT bahwa kita menggunakan sensor tipe `DHT11` (bukan `DHT22` atau `DHT21`).
- `#define RAIN_PIN 22`: Memberi nama `RAIN_PIN` untuk pin `22`, yang terhubung ke pin **DO (Digital Out)** dari modul sensor hujan.
- `#define LDR_PIN 35`: Memberi nama `LDR_PIN` untuk pin `35`, yang terhubung ke pin **DO (Digital Out)** dari modul sensor LDR.

2.4.2 Timer Non-Blocking (Konsep Penting!)

Lokasi: Tepat setelah blok "KONFIGURASI SENSOR" di atas.

```
unsigned long previousMillis = 0;  
const long interval = 2000; // Baca data setiap 2 detik
```

Penjelasan Kode:

- `unsigned long previousMillis = 0;`: Membuat variabel untuk menyimpan "kapan terakhir kali kita membaca sensor". `unsigned long` adalah tipe data yang bisa menyimpan angka sangat besar (waktu `millis()` akan terus bertambah). Kita set ke `0` agar sensor langsung dibaca saat pertama kali program menyala.
- `const long interval = 2000;`: Membuat konstanta yang menyimpan interval pembacaan sensor, yaitu `2000` milidetik (atau 2 detik).

2.4.3 Inisialisasi Objek Sensor

Lokasi: Di bagian objek global, setelah `TFT_eSPI tft = ...`

```
DHT dht(DHTPIN, DHTTYPE);
```

Penjelasan Kode:

- `DHT dht(DHTPIN, DHTTYPE);`: Ini adalah "konstruktor". Kita membuat sebuah objek bernama `dht` dari *class* `DHT`. Kita memberinya dua parameter yang sudah kita tentukan sebelumnya: `DHTPIN` (pin 21) dan `DHTTYPE` (`DHT11`). Sekarang, kita bisa menggunakan `dht.` untuk mengakses semua perintah di *library* itu.

2.4.4 Pembaruan di `setup()`

Lokasi: Di dalam `void setup()`, sisipkan kode ini **setelah** `tft.setRotation(1)` dan **sebelum** `lv_init()`.

```
pinMode(RAIN_PIN, INPUT);  
pinMode(LDR_PIN, INPUT);  
dht.begin();
```

Penjelasan Kode:

- `pinMode(RAIN_PIN, INPUT);`: Memberi tahu mikrokontroler (ESP32) bahwa pin yang kita beri nama `RAIN_PIN` (pin 22) akan digunakan sebagai `INPUT` (menerima data).
- `pinMode(LDR_PIN, INPUT);`: Sama, memberi tahu mikrokontroler bahwa `LDR_PIN` (pin 35) akan digunakan sebagai `INPUT`.
- `dht.begin();`: "Menyalakan" atau menginisialisasi komunikasi dengan sensor DHT. Ini harus dipanggil sekali di `setup()` sebelum kita bisa membaca data darinya.

2.5 Pengiriman dan Penampilan Data pada LVGL

Disini kita akan membuat fungsi baru yaitu `read_sensors()`. Tujuannya adalah untuk mengelompokkan semua logika sensor kita mulai dari Sensor DHT11, Raindrop, dan LDR. Sebagai fungsi baru, lokasinya berada setelah `setup()` berakhir dan sebelum `void loop()` dimulai.

```
void read_sensors() {
    // Kita akan menulis program kita disini
}
```

2.5.1 Bagian Timer (Gerbang Logika)

Kode ini adalah "gerbang" yang memastikan pembacaan sensor hanya terjadi setiap 2 detik.

```
unsigned long currentMillis = millis();
if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;
    // Selanjutnya kita akan menulis program kita disini
}
```

Penjelasan Kode:

- `unsigned long currentMillis = millis();`: Mendapatkan waktu (jumlah milidetik) saat ini sejak ESP32 menyala.
- `if (currentMillis - previousMillis >= interval)`: Ini adalah inti *timer non-blocking*.
 - Contoh: Waktu saat ini (5000ms) - Terakhir baca (3000ms) = 2000ms.
 - Apakah 2000ms >= 2000ms? Ya.
 - Maka, jalankan semua kode di dalam `{...}`.
 - Jika belum 2 detik, `if` ini `false` dan semua kode di dalamnya (Bagian 5B, 5C, 5D) akan **dilewati (di-skip)**.
- `previousMillis = currentMillis;`: **Sangat Penting!** Segera setelah kita lolos `if`, kita *reset* timernya dengan menyimpan waktu saat ini sebagai "waktu terakhir baca".

2.5.2 Bagian Sensor DHT11 (Suhu & Kelembapan)

Setelah lolos gerbang timer, ini adalah hal pertama yang kita lakukan.

```
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();

if (isnan(temperature) || isnan(humidity)) {
```

```

    Serial.println("Gagal membaca DHT11!");
} else {
    Serial.printf("Suhu:  %.1f  °C  |  Kelembapan:  %.1f  %%\n",
temperature, humidity);

    if (objects.label_temp != NULL) {
        char tempString[16];
        sprintf(tempString, "%.1f °C", temperature);
        lv_label_set_text(objects.label_temp, tempString);
    }
    if (objects.label_humadity != NULL) {
        char humString[16];
        sprintf(humString, "%.1f %%", humidity);
        lv_label_set_text(objects.label_humadity, humString);
    }
}
}

```

Penjelasan Kode:

- `float temperature = dht.readTemperature();`: Membaca suhu dari sensor. Hasilnya adalah angka desimal (`float`).
- `float humidity = dht.readHumidity();`: Membaca kelembapan.
- `if (isnan(temperature) || isnan(humidity))`: Cek error. `isnan` ("Is Not a Number") akan `true` jika sensor dicabut atau gagal dibaca. `||` artinya "atau".
- `Serial.println(...)`: Mencetak pesan error ke Serial Monitor jika pembacaan gagal.
- `else`: Jika tidak ada error (pembacaan berhasil)...
- `Serial.printf(...)`: Cetak data ke Serial Monitor untuk *debugging*. `%.1f` adalah format untuk `float` dengan 1 angka di belakang koma.
- `if (objects.label_temp != NULL)`: `objects.label_temp` adalah nama label dari `ui.h` (SquareLine). Ini adalah cek keamanan untuk memastikan label itu ada.
- `char tempString[16];`: Membuat "kotak" (array) karakter untuk menampung teks, ukurannya 16 karakter.
- `sprintf(tempString, "%.1f °C", temperature);`: "Print ke string". Ini mengambil `float` (misal: 27.5), memformatnya menjadi teks "27.5 °C", dan menyimpannya di `tempString`.
- `lv_label_set_text(objects.label_temp, tempString);`: **Ini Jembatannya!** Perintah LVGL untuk mengambil `objects.label_temp` dan mengganti teksnya dengan apa pun yang ada di `tempString`.
- Blok `if (objects.label_humadity ...)` adalah logika yang sama persis, tapi untuk label kelembapan.

2.5.3 Bagian Sensor Hujan (Digital)

Ini adalah sensor kedua yang kita baca.

```
int rainValue = digitalRead(RAIN_PIN);
```

```

const char *rainStatus = (rainValue == LOW) ? "Ya" : "Tidak";
Serial.printf("Nilai Sensor Hujan (Digital): %d | Status Hujan: %s\n", rainValue, rainStatus);

if (objects.label_rain != NULL) {
    lv_label_set_text(objects.label_rain, rainStatus);
}

```

Penjelasan Kode:

- `int rainValue = digitalRead(RAIN_PIN);`: Membaca pin digital hujan. `rainValue` akan berisi `LOW` (atau 0) jika basah, atau `HIGH` (atau 1) jika kering.
- `const char *rainStatus = (rainValue == LOW) ? "Ya" : "Tidak";`: Ini adalah "operator ternary" (if/else singkat).
 - `(rainValue == LOW)`: Apakah nilai sensor `LOW` (basah)?
 - `? "Ya"`: Jika YA, set `rainStatus` menjadi teks "Ya".
 - `: "Tidak"`: Jika TIDAK (berarti `HIGH`), set `rainStatus` menjadi teks "Tidak".
- `lv_label_set_text(objects.label_rain, rainStatus);`: Memperbarui label hujan di UI LVGL dengan teks "Ya" atau "Tidak".

2.5.4 Bagian Sensor LDR (Cahaya Digital)

Sensor terakhir yang kita baca.

```

int ldrValue = digitalRead(LDR_PIN);
const char *lightStatus = (ldrValue == LOW) ? "Terang" : "Gelap";
Serial.printf("Nilai LDR (Digital): %d | Status Cahaya: %s\n", ldrValue, lightStatus);

if (objects.label_light != NULL) {
    lv_label_set_text(objects.label_light, lightStatus);
}

```

Penjelasan Kode:

- `int ldrValue = digitalRead(LDR_PIN);`: Membaca pin digital LDR. `LOW` (0) jika terang, `HIGH` (1) jika gelap (tergantung modulnya, kita ikuti logika kode).
- `const char *lightStatus = (ldrValue == LOW) ? "Terang" : "Gelap";`: Logika ternary yang sama. Jika `LOW` (terang), set teks ke "Terang". Jika `HIGH` (gelap), set teks ke "Gelap".
- `lv_label_set_text(objects.label_light, lightStatus);`: Memperbarui label cahaya di UI LVGL.

2.5.5 Pembaruan di `loop()`

Setelah selesai membuat fungsi `read_sensors()`, kita akan menambahkan panggilan `read_sensors()` pada `void loop()`.

```

void loop() {
    lv_timer_handler(); // [LEVEL 1] Jalankan LVGL

```



```
read_sensors();    // [LEVEL 2] Panggil fungsi baca sensor
delay(3);
}
```

Penjelasan Kode:

- `lv_timer_handler()`: Tetap menjadi prioritas utama. Ini adalah "detak jantung" LVGL, mengurus semua animasi dan penggambaran ulang.
- `read_sensors()`: Memanggil fungsi yang baru saja kita buat. Fungsi ini akan berjalan sangat cepat, hanya mengecek `if (millis()...)` dan langsung keluar jika belum waktunya.
- `delay(3)`: Jeda singkat (3 milidetik) untuk memberi "napas" pada ESP32.

2.6 Kesimpulan

Chapter 2 ini membahas secara komprehensif proses perakitan, pemrograman, dan integrasi sistem stasiun cuaca mini berbasis IoT menggunakan LVGL. Materi diawali dengan penjelasan konsep dasar sistem IoT serta alur data antara sensor, mikrokontroler, dan antarmuka grafis. Selanjutnya, dijelaskan tahapan perakitan perangkat keras meliputi sensor DHT11, LDR, dan Raindrop, serta kebutuhan perangkat lunak seperti Arduino IDE, LVGL Library, dan EEZ Studio.

Bagian pemrograman menjelaskan langkah-langkah inisialisasi sensor, struktur kode program, hingga mekanisme pembacaan data secara berkala menggunakan fungsi `millis()` agar sistem tetap responsif tanpa menghambat GUI. Integrasi data sensor dengan LVGL dilakukan melalui fungsi `lv_label_set_text()` yang mengirimkan hasil pengukuran suhu, kelembaban, intensitas cahaya, dan deteksi hujan langsung ke elemen tampilan grafis secara real-time.

Melalui chapter ini, peserta diharapkan memahami bagaimana sensor lingkungan dihubungkan, diprogram, dan divisualisasikan secara interaktif menggunakan ESP32 dan LVGL. Dengan demikian, mereka memperoleh keterampilan praktis dalam merancang sistem pemantauan cuaca yang real-time, efisien, dan mudah dikembangkan untuk berbagai aplikasi berbasis IoT.

DAFTAR PUSTAKA

- Herdianzenda, R., Siswanto, A., & Istiadi, Y. (2021). Rancang bangun stasiun cuaca mini berbasis mikrokontroler untuk pemantauan lingkungan. *Jurnal Teknologi Informasi dan Komputer*, 7(2), 101–110.
- IPCC. (2022). *Climate change 2022: Impacts, adaptation, and vulnerability. Contribution of Working Group II to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change*. Cambridge University Press.
- Muhammad, U., Irvawansyah, & Muhammad, D. (2023). ENVIRONMENTAL CONDITION MEASUREMENT SYSTEM WITH A MINI WEATHER STATION USING ESP32. *Media Elektrik Journal*, 21. <https://doi.org/10.59562/metrik.v21i1.657>
- Putra, D. Y., & Kusuma, H. (2020). Implementasi Internet of Things (IoT) pada sistem monitoring lingkungan berbasis web. *Jurnal Teknologi Elektro*, 11(1), 55–62.
- Safriadi, D. N. A. P. (2024). Implementation of LVGL and LovyanGFX into a Portable Datalogger Embedded System. *Journal of Computation Physics and Earth Science*, 4, 62-68. <https://doi.org/10.63581/JoCPES.v4i2.05>
- Sani, A., & Firdaus. (2021). Stasiun Pemantau Cuaca Berbasis IoT (Internet of Things) dengan Metode Exponential Smoothing. *JOURNAL OF APPLIED ELECTRICAL ENGINEERING*, 5, 50-54. <https://doi.org/10.30871/jaee.v5i2.3172>
- Wahyudi, B., Nugroho, S., & Prasetyo, A. (2021). Sistem pemantauan suhu dan kelembaban berbasis IoT untuk mendukung sektor agrikultur. *Jurnal Teknologi dan Sistem Komputer*, 9(3), 45–52

TEROBOS